

# Confidential Transfers on Sui

## Draft Version 0.2

Mysten Labs

October 6, 2026

### Abstract

We formalize the confidential-transaction mechanism implemented in the accompanying repository as a cryptographic protocol for a public ledger. The construction allows users to convert public coins into encrypted balances, transfer value privately between registered parties, and recover public coins from private balances, while preserving ledger-level verifiability. An optional auditing layer equips a designated auditor with decryption capability over each confidential transfer. The protocol combines Twisted ElGamal encryption [CMTA20] over Ristretto255 [dVGH<sup>+</sup>23] with non-interactive zero-knowledge proofs for balance conservation, ciphertext consistency, range validity, and balance re-keying under rotated keys. We describe the participating roles, the on-chain state, and the core transition protocols, and we model security in the universal-composability framework [Can01], proving that the protocol realizes an ideal functionality that captures its integrity and privacy guarantees.

## 1 Introduction

Confidential transactions aim to reconcile two requirements that are naturally in tension on a public blockchain. On the one hand, the ledger must enforce correctness: no user should be able to create value, overspend a balance, or violate the supply relationship between public and private representations of an asset. On the other hand, transfer amounts and confidential balances should remain hidden from observers. The system implemented in this repository realizes this objective for Sui-based assets by maintaining encrypted balances on-chain and requiring every state transition that posts fresh ciphertexts to be accompanied by efficiently verifiable zero-knowledge proofs; the remaining transitions (public wraps and homomorphic merges of pending deposits) are publicly checkable and carry none.

At a high level, a public coin of type  $T$  may be deposited into a backing pool and represented as confidential balance under a user’s public key. Once in private form, value can be transferred between registered accounts without revealing the amount to the ledger. A holder may later withdraw part of a confidential balance back into a public coin. The central invariant is that the total encrypted value represented on-chain remains synchronized with the public assets held in the pool; the corresponding privacy goal is that observers learn no more about confidential balances and private transfers than what is already implied by public wrap and unwrap events together with the visible transaction graph. The protocol also supports an optional auditing layer, in which an issuer-maintained auditor key can decrypt the amounts of the transfers that carry handles under it. An account’s balance is reconstructed by accumulating the transfers the auditor can read rather than decrypted outright.

## 2 Model and Cryptographic Tools

Let  $\mathbb{G}$  denote the Ristretto255 prime-order group of order  $q$  [dVGH<sup>+</sup>23], and let  $G, H \in \mathbb{G}$  be fixed generators with unknown discrete-log relation, where  $G$  is the standard base point and  $H$  is obtained by hashing a domain-separated label to the curve. Each confidential account is associated with a key pair  $(\text{sk}, \text{pk})$ , where  $\text{sk} \xleftarrow{\$} \mathbb{Z}_q$  and  $\text{pk} = \text{sk} \cdot G$ .

The protocol uses Twisted ElGamal encryption [CMTA20]. For a message  $m \in \mathbb{Z}_q$  and randomness  $r \xleftarrow{\$} \mathbb{Z}_q$ , encryption under public key  $\text{pk}$  is defined by

$$\text{Enc}(\text{pk}, m; r) = (C, D) = (rG + mH, r \cdot \text{pk}).$$

The first component is a Pedersen commitment [Ped92] to  $m$  with blinding  $r$ , and the second component is a *decryption handle* tied to the recipient's public key. The scheme is additively homomorphic: component-wise addition of ciphertexts corresponds to addition of plaintexts and blindings. In addition, the implementation uses the trivial encryption  $\text{TrivEnc}(m) = (mH, \mathcal{O})$ , where  $\mathcal{O}$  is the identity of  $\mathbb{G}$ , to embed public values into the encrypted domain. Decryption proceeds by observing that

$$C - \text{sk}^{-1} \cdot D = mH.$$

The implementation restricts balances so that the resulting discrete logarithm remains computationally feasible to recover via a baby-step/giant-step procedure over a bounded domain. To represent a 64-bit balance while preserving efficient range checks, the protocol decomposes a value  $v$  into four 16-bit limbs,

$$v = v_0 + 2^{16}v_1 + 2^{32}v_2 + 2^{48}v_3, \quad v_i \in [0, 2^{16}),$$

and encrypts each limb independently. We write

$$\text{EA}(\text{pk}, v) = (E_0, E_1, E_2, E_3), \quad E_i = \text{Enc}(\text{pk}, v_i; r_i).$$

We write  $\text{EA}(\text{pk}, v; \rho)$  for the same construction with explicit per-limb blindings  $\rho = (r_0, \dots, r_3)$ . Whenever the system needs a single encryption of the represented value, it homomorphically collapses the four limbs according to

$$\text{Collapse}(\text{EA}) = E_0 + 2^{16}E_1 + 2^{32}E_2 + 2^{48}E_3.$$

The protocol also uses a *multi-recipient* extension of Twisted ElGamal, in which one commitment is shared by several decryption handles. For a message  $m \in \mathbb{Z}_q$ , a blinding  $r \xleftarrow{\$} \mathbb{Z}_q$ , and a tuple of recipient public keys  $(\text{pk}_1, \dots, \text{pk}_n)$ ,

$$\text{MREnc}((\text{pk}_1, \dots, \text{pk}_n), m; r) = (C, (D_j)_{j=1}^n), \quad C = rG + mH, \quad D_j = r \cdot \text{pk}_j.$$

The Pedersen commitment  $C$  is shared across all recipients; only the decryption handles differ. Any recipient  $j$  recovers  $mH$  by computing  $C - \text{sk}_j^{-1}D_j$ , and the marginal cost of adding a recipient is a single group element.  $\text{MREnc}_j(\cdot)$  denotes the single-recipient ciphertext  $(C, D_j)$  induced by projecting  $\text{MREnc}(\cdot)$  onto recipient  $j$ . A confidential transfer reuses one set of commitments for three readers.

The protocol relies on four Fiat–Shamir [FS87] NIZKs over  $\mathbb{G}$ , modeled in the security analysis (Section 6.1) as  $\mathcal{F}_{\text{NIZK}}$  instances for the relations below; full constructions and proofs are in Appendix A. We write  $\Pi(\cdot; s)$  for a proof on statement  $s$  whose witness precedes the semicolon.

*DDH*  $\Pi_{\text{DDH}}$  (Appendix A.2, Figure 3) is the Chaum–Pedersen [CP93] proof of knowledge of a shared discrete logarithm,

$$\mathcal{R}_{\text{DDH}} = \{ ((E, F, X, Y); x) : X = xE, Y = xF \}.$$

It enforces every balance-preserving update: a difference ciphertext  $\Delta$  encrypts zero under  $\text{pk} = \text{sk}G$  iff  $\Delta.D = \text{sk} \cdot \Delta.C$ , certified by  $\Pi_{\text{DDH}}(\text{sk}; G, \Delta.C, \text{pk}, \Delta.D)$ . An accepted proof on such a  $\Delta$ , combined with the on-chain check  $\text{pk} \neq \mathcal{O}$  enforced at registration and key rotation, forces the encrypted difference to be zero; the full no-inflation argument, including why the nonzero check is essential, is given in the UC realization (Section 6.3). Key rotation uses a batched generalization of this proof,  $\Pi_{\text{rekey}}$  (below), to re-key every balance handle at once under the shared witness  $w = \text{sk}'\text{sk}^{-1}$ .

*ElGamal PoK*  $\Pi_{\text{EG}}$  (Appendix A.3, Figure 4) is a proof of knowledge of an opening of one Twisted ElGamal ciphertext,

$$\mathcal{R}_{\text{EG}} = \{ ((\text{pk}, C, D); (m, r)) : C = rG + mH, D = r\text{pk} \}.$$

Unlike  $\Pi_{\text{DDH}}$  in the balance relation, this proof is used purely for extraction by the simulator and requires no nonzero condition:  $C$  is a Pedersen commitment, so its opening is unique under the discrete-log assumption, independently of the public key  $\text{pk}$ .

Ciphertexts sharing one public key admit *witness-folding* (Appendix B):  $t$  same-key instances collapse to a single folded proof  $\Pi_{\text{EG}}^{\otimes}$  for the conjunction relation

$$\mathcal{R}_{\text{EG}}^{\otimes} = \{ ((\text{pk}, (C_j, D_j)_{j=1}^t); (m_j, r_j)_{j=1}^t) : C_j = r_jG + m_jH, D_j = r_j\text{pk} \text{ for all } j \},$$

whose size  $(2\mathbb{G} + 2\mathbb{F})$  is independent of  $t$ ;  $\Pi_{\text{EG}}$  is the case  $t = 1$ . Lemma 3 shows the folded proof is an unconditional NIZK proof of knowledge for  $\mathcal{R}_{\text{EG}}^{\otimes}$ , so it instantiates the corresponding  $\mathcal{F}_{\text{NIZK}}$  relation wherever the analysis expects the per-ciphertext proofs. The algorithms in Section 5 use one  $\Pi_{\text{EG}}^{\otimes}$  instance per key in every operation carrying multiple same-key ciphertexts.

*Range*  $\Pi_{\text{range}}$  is a proof of knowledge of bounded openings of a list of Pedersen commitments (the limb components  $E.C$ ),

$$\mathcal{R}_{\text{range}}^w = \{ ((C_\ell)_\ell; (m_\ell, r_\ell)_\ell) : C_\ell = r_\ell G + m_\ell H, m_\ell \in [0, 2^w] \}.$$

Knowledge soundness, not plain soundness, is required: Pedersen commitments are perfectly hiding, so  $\{C : \exists(m, r), C = rG + mH \wedge m \in [0, 2^w]\}$  is the trivial language  $\mathbb{G}$ , and only an extractor of the prover’s opening pins a value. We instantiate it with Bulletproofs [BBB<sup>+</sup>18], at  $w = 16$  over balance and amount limbs, the only width the protocol proves. The 32-bit auditor chunks of Section 3.3 carry no range proof of their own: each is a fixed linear combination of two limbs this proof already bounds.

*Re-keying*  $\Pi_{\text{rekey}}$  (Appendix A.4, Figure 5) is a batched Chaum–Pedersen proof of knowledge of a single scalar  $w$  that relates the old and new keys and re-keys every balance handle,

$$\mathcal{R}_{\text{rekey}} = \{ ((\text{pk}_U, \text{pk}'_U, (D_i, D'_i)_{i=0}^3); w) : \text{pk}'_U = w \text{pk}_U, D'_i = w D_i \text{ for } i = 0, \dots, 3 \}.$$

Key rotation (Algorithm 8) keeps the four limb commitments  $E_i.C$  and replaces only the handles, so the witness  $w = \text{sk}'_U \text{sk}_U^{-1}$  carries each old handle  $D_i$  to its counterpart  $D'_i$  under the new key  $\text{pk}'_U$ , which the ledger additionally checks is nonzero. As with  $\Pi_{\text{DDH}}$ , the check  $\text{pk}'_U \neq \mathcal{O}$  ensures that the re-keyed limbs still encrypt the original values.

**Definition 1.** An encrypted amount  $\text{EA} = (E_0, E_1, E_2, E_3)$  under public key  $\text{pk} \neq \mathcal{O}$  is well-formed if each limb plaintext lies in  $[0, 2^{16})$  and each limb is a valid Twisted ElGamal encryption under  $\text{pk}$ . On-chain, the ledger checks this from submitted proofs: a range proof  $\pi_{\text{range}}$  for  $(E_i.C)_{i=0}^3$  and a single witness-folded consistency proof  $\gamma$  (one  $\Pi_{\text{EG}}^{\otimes}$  instance covering the four limbs  $(E_i)_{i=0}^3$  under  $\text{pk}$ ), which the holder generates off-chain and includes in the transaction.

## 3 Protocol Description

We first describe the parties and state maintained by the protocol, then the transfer and auditing flows. The state transitions of this section appear as pseudocode in Section 5, with variables and primitives explained inline as comments.

### 3.1 Parties and State

**Parties.** The protocol involves four kinds of parties. The *ledger* stores all confidential state and verifies the accompanying proofs. The *coin issuer*, the party controlling the public token type  $T$ , creates the confidential token object associated with  $T$ , maintains the on-chain auditor key described below, and may impose policy restrictions or emergency controls such as freezing. *Account holders* each register a public key for a given confidential asset and may thereafter hold encrypted balances, receive confidential transfers, and convert between the public and private representations of the asset. The *auditor*, associated with an encryption key pair  $(\text{sk}_{\text{aud}}, \text{pk}_{\text{aud}})$ , holds no confidential balance; its sole role is to read the amounts of the transfers that carry handles under its key, through the mechanism of Section 3.3. A configuration with no auditor is a valid instantiation and reduces to the baseline protocol.

**State.** For each token type  $T$ , the confidential system maintains a shared pool containing the public coins that back all confidential balances. The issuer also maintains, scoped to  $T$ , an on-chain auditor slot holding the current auditor key  $\text{pk}_{\text{aud}}$  and an outgoing key  $\text{pk}_{\text{aud}}^{\text{old}}$  that remains acceptable until the issuer’s next update; either key may be  $\perp$ , and  $\text{pk}_{\text{aud}} = \perp$  means auditing is off. Each registered account holder maintains an active encrypted balance, represented as a four-limb encrypted amount under that holder’s public key  $\text{pk}$ , which is chosen per token. Upon registration this active balance is initialized to a well-formed encryption of zero, and therefore carries bound counter  $\tau_{\text{act}} = 1$  from the outset. The account stores no auditor material of any kind: auditability is carried entirely by the handles attached to individual transfers. In addition, the account stores pending deposits in two forms: a cumulative encrypted amount representing confidential transfers received since the last merge, and a public accumulator representing coins wrapped directly into the account.

Registration does not require a proof of knowledge of the account secret key; the ledger checks only that  $\text{pk} \neq \mathcal{O}$ . Solana [Sol24] instead verifies a pubkey-validity proof at account configuration, since its security definitions [Sol21b] decrypt registered accounts under secret keys deposited at registration. Our proof never decrypts under a registered key: credits are fixed by the sender’s validity proofs, and the spend proof carries the spender’s secret key as witness, so knowledge of the key is shown where it is used. A key with unknown discrete log can receive but neither spend nor decrypt, harming only its owner; forgoing the proof also enables third-party registration under a default key.

**Bound counter.** To keep merged balances within the bounded discrete-log domain, each encrypted aggregate carries an integer *bound counter*  $\tau$  that counts the number of **u16**-bounded values folded into it. Each limb plaintext is therefore bounded by  $\tau \cdot (2^{16} - 1)$ , so maintaining  $\tau \leq 2^{16} - 1$  keeps every limb below  $2^{32}$  and therefore inside the baby-step/giant-step recovery domain. The ledger maintains the bound through the following rules, referred to below as the *counter invariant*.

**Counter invariant.**

- A fresh empty aggregate has  $\tau = 0$ ; a freshly verified well-formed encryption has  $\tau = 1$ . Every operation that installs a fresh range-proven active balance (registration, transfer, unwrap, update balance) leaves  $\tau_{\text{act}} = 1$ , and each incoming credit increments the receiver's  $\tau_{\text{pnd}}$  by one. Merge adds  $\tau_{\text{pnd}}$  to  $\tau_{\text{act}}$ , plus one if the public accumulator is nonzero, and clears the pending state.
- A credit to receiver  $R$  is admitted only if  $\tau_{\text{act}}^R + \tau_{\text{pnd}}^R + 2 < 2^{16}$ : one slot for the credit and one reserved for the eventual merge of the public accumulator. The reserved slot is claimed by the first wrap since the last merge, so while the accumulator is nonzero further wraps consume no new slot, whereas a wrap onto an empty accumulator is subject to the same condition.
- Merge and update balance (Section 3.2) are independent: either may be invoked on its own and in either order. Update balance reads and rewrites only the active balance and never raises  $\tau_{\text{act}}$ , so it preserves or decreases the joint counter  $\tau_{\text{act}}^R + \tau_{\text{pnd}}^R$  and the reserved-slot accounting remains sound no matter when, or with what pending state, it is called.

The separation between active and pending state is essential. Incoming transfers do not immediately alter the recipient's active balance. Instead, they are accumulated and later merged in one homomorphic step. This design allows the sender's transfer proof to remain local to the sender.

### 3.2 Confidential Transfers

**Public-to-private conversion.** Suppose a user wishes to *wrap* a public coin of value  $v$  into confidential balance for some registered receiver  $R$ . The ledger deposits the public coin into the shared pool associated with the confidential asset and increments the receiver's pending public accumulator by  $v$ . No encryption or proof accompanies this step: the same public value  $v$  that enters the pool is recorded in the accumulator, so the ledger itself enforces equality between the public and private representations. The resulting value is not yet part of the receiver's active confidential balance; it enters the active state only after a later merge step.

**Private transfer.** A private transfer moves confidential value from a single sender  $S$  with public key  $\text{pk}_S$  and active encrypted balance  $B_S$  to a list of registered receivers  $R_1, \dots, R_K$  with public keys  $\text{pk}_{R_k}$  and per-receiver amounts  $a_k$ . The single-receiver transfer is the special case  $K = 1$ .

The sender first decrypts  $B_S$  locally, computes the remaining balance  $b'_S = b_S - \sum_{k=1}^K a_k$ , and re-encrypts it as a well-formed four-limb ciphertext  $B'_S = \text{EA}(\text{pk}_S, b'_S)$ . It then samples a fresh ephemeral scalar  $e$ , publishes the point  $P = eG$ , and forms the sender-side Diffie–Hellman secret  $Z_S = e \text{pk}_S = \text{sk}_S P$ . For each receiver  $R_k$ , the sender derives its per-limb blindings  $\rho_{k,i} = \text{KDF}(Z_S, \cdot)$  from  $Z_S$  using a key-derivation hash KDF (a domain-separated hash, modeled as a random oracle in the security analysis; concrete implementation uses two-stage HKDF) and encrypts the corresponding transfer amount as  $A_{R_k} = \text{EA}(\text{pk}_{R_k}, a_k; \rho_k)$ . Deriving the blindings from  $Z_S$  lets the sender reconstruct every  $\rho_{k,i}$  from  $Z_S = \text{sk}_S P$  and read each  $a_k$  back off the on-chain receiver commitments, so no separate sender-side amount ciphertext need be posted. Because the twisted-ElGamal commitment is independent of the public key (Section 8), the sender's aggregate debit reuses the very same commitments: writing  $r_S = \sum_{k,i} 2^{16i} \rho_{k,i}$  for the collapsed blinding, the aggregate

sender-side ciphertext is

$$\hat{A}_S = (\hat{A}_S.C, \hat{A}_S.D), \quad \hat{A}_S.C = \sum_{k=1}^K \text{Collapse}(A_{R_k}).C, \quad \hat{A}_S.D = r_S \text{pk}_S,$$

whose commitment the ledger reconstructs from the receiver commitments; only the single aggregate handle  $\hat{A}_S.D$  is posted.

Callers may attach an optional opaque memo to wrap and transfer operations, recorded verbatim in the public log; we omit memos from the cryptographic description below. The sender submits  $B'_S$ , the ephemeral point  $P$ , the receiver ciphertexts  $(A_{R_k})_{k=1}^K$ , the aggregate handle  $\hat{A}_S.D$ , and proofs as transaction arguments. First, the sender attaches well-formedness material for  $B'_S$  and for every  $A_{R_k}$ : one batched range proof covers all  $4(K+1)$  limb commitments,<sup>1</sup> and the ElGamal consistency proofs are witness-folded per key (Appendix B). A single folded proof  $\gamma_S$  under  $\text{pk}_S$  covers the four limbs of  $B'_S$  together with the aggregate  $\hat{A}_S = (\hat{A}_S.C, \hat{A}_S.D)$ , certifying in particular that  $\hat{A}_S$  is a well-formed Twisted ElGamal encryption under  $\text{pk}_S$  of the total  $\sum_k a_k$  with some aggregate blinding the sender knows; one folded proof  $\gamma_{R_k}$  under each  $\text{pk}_{R_k}$  covers that receiver's four limbs. Second, the sender includes a single DDH proof that the old balance equals the new balance plus the total transferred amount. Third, when the asset has an auditor, the sender attaches the per-transfer auditor handles and the single folded proof described in Section 3.3. Writing

$$\Delta = \text{Collapse}(B'_S) + \hat{A}_S - \text{Collapse}(B_S),$$

the intended relation is that  $\Delta$  encrypts zero under  $\text{pk}_S$ . This is certified by a DDH proof showing that the pair  $(\Delta.C, \Delta.D)$  is consistent with secret key  $\text{sk}_S$  relative to  $(G, \text{pk}_S)$ .

Per-pair binding “the sender debited  $a'_k$  on behalf of  $R_k$ ” is *not* enforced; the ledger only ties the totals. The ledger reconstructs the aggregate commitment

$$\hat{A}_S.C = \sum_{k=1}^K \text{Collapse}(A_{R_k}).C$$

directly from the receiver commitments, which together with the range proofs over each  $A_{R_k}$  (pinning every  $a_k$  in  $[0, 2^{16})$  per limb, and so  $\sum_k a_k$  in  $\mathbb{Z}$  below the group order) and the sender-side folded consistency proof  $\gamma_S$  forces the aggregate sender-side debit to equal  $\sum_k a_k$  exactly.

**Private-to-public conversion.** To withdraw, or *unwrap*, a public amount  $v$  from a confidential balance, a holder  $U$  with public key  $\text{pk}_U$  decrypts the current active balance, subtracts  $v$  locally, and re-encrypts the resulting amount as a well-formed ciphertext  $B'_U = \text{EA}(\text{pk}_U, b'_U)$ . The holder then considers the difference

$$\Delta = \text{Collapse}(B'_U) + \text{TrivEnc}(v) - \text{Collapse}(B_U).$$

If the claimed withdrawal is correct, then  $\Delta$  encrypts zero under  $\text{pk}_U$ . The holder therefore submits  $B'_U$ , the public value  $v$ , a range proof and a witness-folded ElGamal consistency proof for  $B'_U$  (as in Definition 1), and a DDH proof certifying that  $\Delta$  is a zero encryption under  $\text{pk}_U$ . Once verified, the ledger updates the active balance to  $B'_U$  and releases  $v$  units of the public coin from the shared pool.

<sup>1</sup>Bulletproofs aggregation covers a power-of-two number of commitments, capped at 32 per proof by the on-chain verifier. The implementation therefore splits the  $4(K+1)$  commitments greedily into power-of-two chunks with one proof per chunk; a single proof suffices exactly when  $4(K+1)$  is itself such a chunk, as for  $K \in \{1, 3, 7\}$ . Each proof is logarithmic in its chunk size, so the split adds a few group elements over a single aggregate, and the soundness argument applies per chunk.

**Merging pending deposits and restoring well-formedness.** Pending deposits are absorbed into the active balance by two ledger operations, *merge* and *update balance*, whose counter accounting is governed by the invariant of Section 3.1. Merge folds the pending balance into the active one using homomorphic additions of encryptions. This preserves the plaintext balance, but not the property that every limb lies in  $[0, 2^{16})$ : repeated homomorphic addition may produce limbs whose plaintexts exceed that range even though the overall balance remains decryptable. A holder may invoke merge multiple times before updating the balance, each merge advancing the bound counter  $\tau$ ; the ledger aborts a pending deposit that would exhaust the remaining slack.

Update balance restores limb well-formedness, and with it the slack that admits further deposits. The holder decrypts the active ciphertext, re-decomposes the plaintext into fresh 16-bit limbs, and re-encrypts under the same public key to obtain a new well-formed encrypted amount  $B'_U$ . The holder then proves that  $B'_U$  represents exactly the same value as the previous active balance by forming

$$\Delta = \text{Collapse}(B'_U) - \text{Collapse}(B_U)$$

and certifying, again with a DDH proof relative to  $(G, \text{pk}_U)$ , that  $\Delta$  encrypts zero. The holder submits  $B'_U$  together with a range proof and a folded consistency proof for those fresh limbs, and the ledger verifies both well-formedness and equality and replaces the active balance by  $B'_U$ , resetting  $\tau_{\text{act}}$  to 1.

In practice, the holder rarely pays for a balance update as a standalone operation. Every transfer and unwrap already supplies a fresh well-formed new balance with its own range proof and folded consistency proof, so a merge followed by a spend in the same transaction lets the spend's proofs double as the balance update, saving one bulletproof aggregate and one consistency proof. The optimistic spend variants of the implementation also compose naturally with this pattern: if a pending deposit lands between the holder's view of state and execution and the post-merge balance proof fails, the merge still commits and the spend leg short-circuits with a failure event instead of aborting the transaction.

### 3.3 Auditing

**Auditor key management.** Each asset  $T$  carries a single auditor key  $\text{pk}_{\text{aud}}$ , which the issuer may replace at any time (Algorithm 2). Because a transfer's handles are formed under whichever key the sender read from the ledger, a rotation that took effect immediately would invalidate transactions already in flight. The update therefore takes, alongside the new key, an issuer-chosen outgoing key  $\text{pk}_{\text{aud}}^{\text{old}}$ , normally the key being replaced: handles under the outgoing key remain acceptable until the issuer's next update, which ends the grace period by clearing or replacing the outgoing slot; no timer expires it on chain. Setting the new key to  $\perp$  stops requiring auditor data on transfers; once the outgoing slot is cleared as well, auditing is fully off and transfers must carry no auditor data at all.

**Per-transfer auditor handles.** The auditor material on a transfer reuses the receiver ciphertexts rather than duplicating them. A receiver amount  $a_k$  is encrypted as four 16-bit limbs, and the sender pairs those limbs into two 32-bit *chunks*. For  $j \in \{0, 1\}$  the chunk blinding and the auditor handle are

$$\tilde{\rho}_{k,j} = \rho_{k,2j} + 2^{16} \rho_{k,2j+1}, \quad \tilde{D}_{k,j} = \tilde{\rho}_{k,j} \text{pk}_{\text{aud}},$$

and the matching chunk commitment  $\tilde{C}_{k,j} = A_{R_k,[2j]} \cdot C + 2^{16} A_{R_k,[2j+1]} \cdot C$  is a linear combination of commitments already on chain, so it is recomputed by the ledger and never posted. It opens to  $\tilde{a}_{k,j} = a_{k,2j} + 2^{16} a_{k,2j+1} < 2^{32}$  under the blinding  $\tilde{\rho}_{k,j}$ . Only the  $2K$  handles are submitted, together with a single witness-folded ElGamal consistency proof  $\gamma_{\text{aud}}$  under  $\text{pk}_{\text{aud}}$ , carrying its own domain

separator, that covers all  $2K$  chunk ciphertexts  $\tilde{E}_{k,j} = (\tilde{C}_{k,j}, \tilde{D}_{k,j})$  at once. The chunks need no range proof of their own: each is built from two limbs the transfer’s 16-bit range proof already bounds, so its opening is below  $2^{32}$  by construction.

The transaction does not name the auditor key it used. The ledger verifies  $\gamma_{\text{aud}}$  against the current key and, failing that, against the outgoing key while one is set, at most two verifications; the key that verified is written into the transfer event as  $\text{pk}_*$  alongside the handles. A transfer must carry auditor data exactly when a current key is installed; when only the outgoing key remains, during its grace period, a transfer may carry data under it or carry none, and when both slots are empty it must carry none. Consistency is enforced by ElGamal consistency rather than a DLEQ against the receiver handles: a DLEQ would also be sound here, since the sender does not know  $\text{sk}_{\text{aud}}$ , but it would cost one proof per receiver instead of one folded proof per transfer (Appendix C).

**Auditor decryption flow.** Auditing is an entirely off-chain, read-only operation. To read a transfer, the auditor takes the handles  $\tilde{D}_{k,j}$  from the transfer event, recomputes each chunk commitment  $\tilde{C}_{k,j}$  from the on-chain receiver commitments, and decrypts  $\tilde{a}_{k,j}$  with  $\text{sk}_{\text{aud}}$  by the same bounded discrete-log procedure used for balance limbs, now over  $[0, 2^{32})$ . The two chunks reassemble into the exact per-leg amount  $a_k = \tilde{a}_{k,0} + 2^{32}\tilde{a}_{k,1}$ , for every receiver of the batch and not merely the account-level total. Because  $\gamma_{\text{aud}}$  was verified on chain, this succeeds for every accepted transfer that carried auditor data.

Balances are recovered by accumulation rather than by direct decryption, since the auditor never holds  $\text{sk}_U$ . Every hidden change to an account’s balance is a transfer leg, whose amount the auditor reads as above; a sender’s debit for a batch is the sum of that batch’s legs. Everything else that moves the balance is already public: wrap and unwrap amounts appear in clear, the pending-to-active split is visible in the merge entries, and the remaining operations preserve value. Folding these forward from the account’s registration therefore reproduces its active and pending balances exactly. This works only for an account whose every confidential transfer, sent or received, was designated to that auditor; an account with a gap in its history, for instance one that transacted before the auditor key was installed, is not auditable this way, and the coverage condition is made precise in Section 6. The per-transaction cost of auditing is thus the  $2K$  handles and the single folded proof; the ephemeral point  $P$  remains on the transfer, but it now serves only the sender’s recovery of its own outgoing amounts.

## 4 Security Discussion

This section gives an informal account of the protocol’s integrity and privacy guarantees; Section 6 establishes them formally as UC-realization of  $\mathcal{F}_{\text{ConfTx}}$ .

**Integrity of balances.** The principal safety property is conservation of value between the encrypted state and the public backing pool, and each operation preserves it by construction. Wrap credits a recipient’s pending state with exactly the public value locked into the pool. A confidential transfer is accepted only when the sender proves that the old active balance equals the new active balance plus the transferred amount, and that the sum of the amounts credited across the receivers equals the amount debited from the sender (Algorithm 5). Unwrap requires a proof that the decrease in encrypted balance matches the public value released from the pool (Algorithm 6). Merge and balance update compose homomorphic addition with a re-encryption proved equal under  $\Pi_{\text{DDH}}$ , and are therefore value preserving (Algorithm 7). Key rotation reuses the existing commitments and only re-keys the decryption handles, so it changes no plaintext (Algorithm 8).

**Non-negativity.** Every balance eligible to become active after transfer, unwrap, or balance update is re-encoded as four limbs, each proved to lie in  $[0, 2^{16})$ , so at the moment of verification the encoded plaintext lies in  $[0, 2^{64})$ . The bound counter  $\tau < 2^{16}$  (Section 3.1) keeps accumulated limbs decryptable between verifications. Together with the balance relation checked at each state transition, no polynomial-time adversary can make an account spend more than its available value without breaking proof soundness or the hardness assumptions underlying the encryption scheme.

**Theorem 1** (Informal integrity statement). *Assuming the discrete-log problem is hard in  $\mathbb{G}$  and the proof systems  $\Pi_{\text{DDH}}$ ,  $\Pi_{\text{EG}}$ ,  $\Pi_{\text{range}}$ , and  $\Pi_{\text{rekey}}$  are sound, no polynomial-time adversary can cause the sum of all confidential balances to exceed the amount of public coin held in the corresponding backing pool, for any auditor keys, including maliciously generated ones, provided the issuer does not invoke the privileged write paths described below.*

**Spending requires owner authentication, not merely  $\text{sk}_U$ .** A transfer, unwrap, or balance update is accepted only if two independent conditions hold: the ledger authenticates the caller as the account’s owner, and the submitted balance, range, and consistency proofs verify. The secret key  $\text{sk}_U$  is relevant only to the second condition. It allows its holder to decrypt the account’s ciphertexts and produce the  $\Pi_{\text{DDH}}$  balance proof, but it carries no authority to invoke an operation on the account: each state transition is tied to the holder’s identity by the ledger; the implementation requires that the transaction sender owns the account. Holding  $\text{sk}_U$  is therefore necessary but not sufficient to spend. In particular, any party that comes to hold  $\text{sk}_U$  gains only a read capability: it can decrypt every balance and transfer amount of the account, but it cannot move funds, since it cannot authenticate as the owner.

**Issuer-controlled escape hatches.** The integrity statement is conditioned on the issuer not exercising a few privileged operations that the implementation exposes for compliance and emergency response and that intentionally bypass conservation: a global pause that disables wraps, transfers, and unwraps; a per-account deny list that prevents an address from sending or receiving; and a direct overwrite of a registered account’s encrypted balance, used to emulate burn or seize. The first two are halting operations and preserve conservation between encrypted state and the backing pool. The overwrite does not: it can leave the sum of confidential balances out of step with the pool, so the issuer must use it only when reconciliation is restored out of band. Holders accept this trust assumption when they register against an issuer’s confidential token.

**Privacy of transactions.** Privacy derives from two sources. Twisted ElGamal is semantically secure under DDH in  $\mathbb{G}$  [CMTA20, ElG85], and its ciphertext component is a perfectly hiding Pedersen commitment; the multi-recipient variant, in which a transfer’s limb commitment carries a receiver handle, a share of the sender’s aggregate handle, and a share of an auditor chunk handle, retains semantic security under the same assumption, since each decryption handle is an independent DDH tuple under the corresponding recipient key. A transfer derives its per-limb blindings deterministically from the sender-side Diffie–Hellman secret  $Z_S = \text{sk}_S P$  through a key-derivation hash KDF, modeled as a random oracle (Section 3.2); under DDH the value  $Z_S$  is pseudorandom to any observer holding neither  $\text{sk}_S$  nor the ephemeral scalar, so that observer cannot query the oracle at  $Z_S$  and the derived blindings are uniformly random in its view, leaving the semantic security above unchanged. The correctness proofs are zero knowledge [FS87, BBB<sup>+</sup>18], so they reveal nothing about the encrypted amount, the blinding randomness, or the holder’s long-term secret beyond the validity of the claimed relation. An observer of a private transfer therefore learns

only that the sender’s encrypted balance was updated consistently and that each receiver obtained a well-formed encrypted amount equal to the sender’s debit; the amounts themselves remain hidden.

The privacy claim is intentionally limited in two ways. Wrap and unwrap amounts are public, since they move value across the boundary between public and private representations, and the transaction graph is visible because the ledger records which accounts interact; the protocol thus hides confidential balances and transfer amounts but not participation patterns or boundary crossings. Second, the party holding the auditor decryption key for an asset learns the amount of every transfer of that asset whose handles designate its key, on both legs. That capability is bounded in two respects: it is a read capability only, since it does not authorize spending, and it does not extend to transfers designating a different key, in particular to one predating the auditor’s installation.

**Theorem 2** (Informal privacy statement). *Assuming DDH in  $\mathbb{G}$ , modeling the transfer key-derivation hash KDF as a random oracle, and assuming the zero-knowledge property of  $\Pi_{\text{DDH}}$ ,  $\Pi_{\text{EG}}$ ,  $\Pi_{\text{range}}$ , and  $\Pi_{\text{rekey}}$ , a polynomial-time observer of the ledger who does not hold an auditor decryption key learns no information about confidential transfer amounts or active confidential balances beyond what follows from the public wrap and unwrap values together with the visible transaction graph. A polynomial-time observer holding  $\text{sk}_{\text{aud}}$  additionally learns the amount of every transfer whose auditor handles verify under  $\text{pk}_{\text{aud}}$ , and, for an account all of whose confidential activity is of that kind, its confidential balance.*

**Confidentiality and corruption.** We call the privacy guarantee against a polynomial-time adversary that holds no auditor decryption key *confidentiality* and treat it as the primary privacy notion. The general case, in which a static adversary also holds an auditor key, does not fold whole accounts into the corruption set: it exposes exactly the transfers designating that key, which is why the ideal functionality can admit statically corrupt auditors directly (Section 6.1) rather than through a re-corruption of their auditees, and why the only cost in the leakage profile is that such a transfer reveals its amount.

The auditor key can change over time: an issuer update (Algorithm 2) may install a corrupt one. Transfers already accepted are unaffected, since their handles remain under the previous key; the corrupt key sees only transfers issued after the update, together with those still designating the outgoing key during its grace period, which lasts until the issuer’s next update. Conversely, rotating away from a leaked key, and clearing it from the outgoing slot, confines the leak to the transfers already carried under it, without any action by holders.

**Malicious auditor keys.** A maliciously generated auditor key cannot break integrity. The key enters a transfer only as the base of its handles and of the chunk proof, whose statements the ledger recomputes homomorphically from limb commitments already fixed by the receiver proofs; the balance relation, the range proofs, and the value-conservation argument of Section 6 never reference it. An adversarial key can therefore never shift value. Its effect is confined to the corrupt-auditor leakage described above and the loss of the audit-read capability. Handles form under any non-identity key, so transfers remain available.

See Section 7 for a comparison with prior security models.

## 5 Formal Description

The following pseudocode uses the primitives from Section 2: `Enc`, `Dec`, `EA`, `Collapse`, `TrivEnc`,  $\Pi_{\text{DDH}}$ ,  $\Pi_{\text{EG}}$ ,  $\Pi_{\text{EG}}^{\otimes}$ ,  $\Pi_{\text{range}}$ , and `Verify`. The map  $\text{TrivEnc}_{\text{limbs}}$  splits a `u64` into four 16-bit limbs and applies

---

**Algorithm 1** Account registration

---

**Require:** Fresh holder key pair  $(\text{sk}_U, \text{pk}_U)$  with  $\text{pk}_U = \text{sk}_U G$ , chosen per token and independent of the account's other tokens. The holder's **Account** object may carry a default key  $\text{pk}_{\text{dft}}$ , which a third party can use to register a token for the account.

**Holder (off-chain)**

- 1: **submit**  $(\text{pk}_U)$   $\triangleright A$  permissionless variant submits no key, and the ledger reads  $\text{pk}_U \leftarrow \text{pk}_{\text{dft}}$  from the **Account**

**Ledger (on-chain)**

- 2: **assert**  $\text{pk}_U \neq \mathcal{O}$
  - 3:  $B_U \leftarrow \text{EA}(\text{pk}_U, 0)$  with bound counter  $\tau_{\text{act}} \leftarrow 1$  ( $\text{EA}(\text{pk}_U, 0)$  is a well-formed encryption);  $B_{\text{pnd}}^U \leftarrow \mathbf{0}$ ,  $\tau_{\text{pnd}} \leftarrow 0$  (encrypted pending balance);  $b_{\text{pub}}^U \leftarrow 0$  (public pending balance)
- 

---

**Algorithm 2** Auditor key update with grace period (issuer)

---

**Require:** Issuer authority over asset  $T$ ; new auditor key  $\text{pk}'_{\text{aud}}$ , or  $\perp$  to disable auditing; outgoing key  $\text{pk}_{\text{aud}}^{\text{old}}$  to remain acceptable during the grace period, normally the key being replaced, or  $\perp$  for none.

**Ledger (on-chain, authorized)**

- 1:  $\text{pk}_{\text{aud}}^{\text{old}} \leftarrow \text{pk}_{\text{aud}}^{\text{old}}$ ;  $\text{pk}_{\text{aud}} \leftarrow \text{pk}'_{\text{aud}}$   $\triangleright$  both slots are issuer-supplied; the grace period lasts until the issuer's next update clears or replaces the outgoing slot—no timer expires it on chain

**Ensure:** Subsequent transfers attach handles under  $\text{pk}_{\text{aud}}$ ; handles under  $\text{pk}_{\text{aud}}^{\text{old}}$  are accepted until the issuer's next update. When  $\text{pk}_{\text{aud}} = \perp$  transfers need carry no auditor data; when additionally  $\text{pk}_{\text{aud}}^{\text{old}} = \perp$ , they must carry none.

---

TrivEnc to each. For a Twisted ElGamal ciphertext  $E = (C, D)$  we write  $E.C$  and  $E.D$  for its components. For a multi-recipient ciphertext  $E = (C, (D_j)_j)$  we write  $E.C$  and  $E.D_j$  accordingly. Write a four-limb ciphertext as  $B = (B_{[0]}, \dots, B_{[3]})$ . When the holder forms  $B = \text{EA}(\text{pk}, v)$ , let  $(v_i, \rho_i)$  denote the openings with  $B_{[i]} = \text{Enc}(\text{pk}, v_i; \rho_i)$ ; when several ciphertexts are in play, the blindings carry the account subscript  $(\rho_{S,i}, \rho_{U,i})$  and a prime carries over from a ciphertext to its limb values, so the openings of  $B'_S$  are  $(v'_i, \rho_{S,i})$ . A superscript on  $\Pi_{\text{range}}$ ,  $\pi_{\text{range}}$ , or  $\text{Verify}_{\text{range}}$  records the bit-width of the proven range: 16 for balance and amount limbs. Lines marked **Holder** run off-chain: the holder uses witnesses to *compute* proof objects (notation  $\pi \leftarrow \Pi_{\dots}$ ). Lines marked **Ledger** run on-chain: the verifier only *checks* submitted ciphertexts and proofs (notation  $\text{Verify}(\dots)$  or **assert**). Proofs are carried as transaction inputs; they are not recomputed by the ledger. A witness-folded consistency proof  $\Pi_{\text{EG}}^{\otimes}((m_j, r_j)_j; \text{pk}, (E_j)_j)$  certifies all listed same-key ciphertexts at once at the cost of a single  $\Pi_{\text{EG}}$ ;  $\text{Verify}_{\text{EG}}^{\otimes}(\gamma; \text{pk}, (E_j)_j)$  denotes its verifier (Appendix B).

Algorithms 1–8 give the protocol; confidential transfer is split across Algorithms 4 and 5 (off-chain holder vs. on-chain ledger).

---

**Algorithm 3** Public-to-private conversion (wrap)

**Require:** Registered recipient  $R$ ;  $v \in \mathbf{u64}$  (value of the public coin consumed);  $b_{\text{pub}}^R > 0$  or  $\tau_{\text{act}}^R + \tau_{\text{pnd}}^R + 2 < 2^{16}$  (the reserved public-merge slot of Section 3.1 is claimed by the first wrap since the last merge; later wraps reuse it).

- 1:  $\text{pool} \leftarrow \text{pool} + v$   $\triangleright$  *public coin locked into the shared backing **Balance** for the asset*
- 2:  $b_{\text{pub}}^R \leftarrow b_{\text{pub}}^R + v$   $\triangleright$  *receiver's pending public accumulator; active ciphertext  $B_R$  unchanged here*

---

---

**Algorithm 4** Private transfer: holder (off-chain)

---

**Require:** Sender keys  $(\text{sk}_S, \text{pk}_S)$ ; receiver keys  $(\text{pk}_{R_k})_{k=1}^K$ ; on-chain active balance  $B_S$ ; amounts  $(a_k)_{k=1}^K$ ; current auditor key  $\text{pk}_{\text{aud}}$  read from the ledger, if auditing is enabled.

- 1:  $b_S \leftarrow \text{Dec}(\text{sk}_S, B_S)$
- 2:  $b'_S \leftarrow b_S - \sum_{k=1}^K a_k$ ;  $B'_S \leftarrow \text{EA}(\text{pk}_S, b'_S)$
- 3:  $e \xleftarrow{\$} \mathbb{Z}_q$ ;  $P \leftarrow eG$ ;  $Z_S \leftarrow e\text{pk}_S \triangleright$  fresh ephemeral; sender-side DH secret  $Z_S = \text{sk}_S P$
- 4: **for**  $k = 1$  **to**  $K$  **do**
- 5:  $\rho_{k,i} \leftarrow \text{KDF}(Z_S, \text{"amt"} \| k \| i)$  for  $i \in \{0, \dots, 3\}$   
 $A_{R_k} \leftarrow \text{EA}(\text{pk}_{R_k}, a_k; \rho_k) \triangleright$  per-limb blindings derived from the DH secret  $Z_S$ , so an honest sender still recovers its own outgoing amounts from  $A_{R_k}$
- 6:  $\tilde{\rho}_{k,j} \leftarrow \rho_{k,2j} + 2^{16}\rho_{k,2j+1}$ ;  $\tilde{D}_{k,j} \leftarrow \tilde{\rho}_{k,j}\text{pk}_{\text{aud}}$  for  $j \in \{0, 1\} \triangleright$  one auditor handle per two limbs: the derived chunk commitment  $C_{k,j} = A_{R_k,[2j]}.C + 2^{16}A_{R_k,[2j+1]}.C$  opens to  $\tilde{a}_{k,j} = a_{k,2j} + 2^{16}a_{k,2j+1} < 2^{32}$  with blinding  $\tilde{\rho}_{k,j}$
- 7: **end for**
- 8:  $r_S \leftarrow \sum_{k,i} 2^{16i}\rho_{k,i}$ ;  $\hat{A}_S.C \leftarrow \sum_{k=1}^K \text{Collapse}(A_{R_k}).C$ ;  $\hat{A}_S.D \leftarrow r_S\text{pk}_S \triangleright$  aggregate sender-side ciphertext: commitment reused from the receiver commitments, only its handle  $\hat{A}_S.D$  is posted
- 9:  $\Delta \leftarrow \text{Collapse}(B'_S) + \hat{A}_S - \text{Collapse}(B_S)$   
 $\pi_{\text{bal}} \leftarrow \Pi_{\text{DDH}}(\text{sk}_S; G, \Delta.C, \text{pk}_S, \Delta.D) \triangleright$  ensures  $\Delta$  encrypts zero: what leaves the sender equals the total credited, so no value is minted
- 10:  $\mathcal{L}_{\text{rng}} \leftarrow (B'_{S,[0]}.C, \dots, B'_{S,[3]}.C)$   
 $\| (A_{R_k,[0]}.C, \dots, A_{R_k,[3]}.C)_{k=1}^K \triangleright 4(K+1)$  limb commitments for one batched range proof; the aggregate  $\hat{A}_S$  reuses the receiver limb commitments and needs no separate entries
- 11:  $\pi_{\text{range}}^{16} \leftarrow \Pi_{\text{range}}^{16}(\text{openings for } \mathcal{L}_{\text{rng}}; \mathcal{L}_{\text{rng}}) \triangleright$  ensures every new-balance and receiver-amount limb is in  $[0, 2^{16})$ , ruling out a negative residual or limb overflow that would fabricate value
- 12:  $\gamma_S \leftarrow \Pi_{\text{EG}}^{\otimes}((v'_i, \rho_{S,i})_{i=0}^3, (\sum_k a_k, r_S); \text{pk}_S, (B'_{S,[i]})_{i=0}^3, \hat{A}_S) \triangleright$  one witness-folded proof for all five sender-keyed ciphertexts: each new-balance limb is well-formed under  $\text{pk}_S$  (the sender can still decrypt its own balance) and the aggregate handle  $\hat{A}_S.D$  is pinned to  $\text{pk}_S$ , not an arbitrary point
- 13: **for**  $k = 1$  **to**  $K$  **do**
- 14:  $\gamma_{R_k} \leftarrow \Pi_{\text{EG}}^{\otimes}((a_{k,i}, \rho_{k,i})_{i=0}^3; \text{pk}_{R_k}, (A_{R_k,[i]})_{i=0}^3) \triangleright$  one folded proof per receiver: every amount limb is well-formed under  $\text{pk}_{R_k}$ , so the recipient can decrypt the funds sent
- 15: **end for**
- 16:  $\gamma_{\text{aud}} \leftarrow \Pi_{\text{EG}}^{\otimes}((\tilde{a}_{k,j}, \tilde{\rho}_{k,j})_{k,j}; \text{pk}_{\text{aud}}, (\tilde{E}_{k,j})_{k,j})$  with  $\tilde{E}_{k,j} = (\tilde{C}_{k,j}, \tilde{D}_{k,j}) \triangleright$  same-key ElGamal consistency under  $\text{pk}_{\text{aud}}$ , with its own domain separator, witness-folded over all  $2K$  chunks into one constant-size proof; no new range proof is needed, since the chunk bound is inherited from  $\pi_{\text{range}}^{16}$
- 17: **submit**  $(B'_S, P, (A_{R_k})_{k=1}^K, \hat{A}_S.D, (\tilde{D}_{k,j})_{k,j}, \pi_{\text{range}}^{16}, \gamma_S, (\gamma_{R_k})_{k=1}^K, \gamma_{\text{aud}}, \pi_{\text{bal}}) \triangleright$  only  $P$  and the single aggregate handle  $\hat{A}_S.D$  are posted, as in the baseline; the  $2K$  auditor handles and one folded proof are what the payload gains, for  $K+2$  consistency proofs in total, one per key and independent of the limb count; the auditor key is not submitted

---

---

**Algorithm 5** Private transfer: ledger (on-chain)

---

**Require:** Transaction inputs from Algorithm 4; on-chain state: the asset is not globally paused; sender account  $S$  with active balance  $B_S$  and public key  $\text{pk}_S$  is registered, not denied, and not frozen; each receiver  $R_k$  with public key  $\text{pk}_{R_k}$ , active counter  $\tau_{\text{act}}^{R_k}$ , and pending encrypted accumulator  $B_{\text{pnd}}^{R_k}$  with counter  $\tau_{\text{pnd}}^{R_k}$  is registered, not denied, not frozen, accepts encrypted deposits, and satisfies  $\tau_{\text{act}}^{R_k} + \tau_{\text{pnd}}^{R_k} + 1 + m < 2^{16}$  where  $m$  is the number of times that receiver appears in the batch ( $m$  slots for the credit, one reserved for the eventual public-pending merge); auditor-key state  $(\text{pk}_{\text{aud}}, \text{pk}_{\text{aud}}^{\text{old}})$  of  $T$ .

- 1: **assert**  $K \geq 1$   $\triangleright$  *nonempty batch*  
 $\triangleright$  *Well-formedness of new balance, per-receiver amounts, and aggregate debit*
- 2: **assert**  $\text{Verify}_{\text{range}}^{16}(\pi_{\text{range}}^{16}; \mathcal{L}_{\text{rng}})$
- 3:  $\hat{A}_S.C \leftarrow \sum_{k=1}^K \text{Collapse}(A_{R_k}).C$ ;  $\hat{A}_S.D \leftarrow$  submitted aggregate handle  $\triangleright$  *commitment reconstructed on-chain from the receiver commitments*
- 4: **assert**  $\text{Verify}_{\text{EG}}^{\otimes}(\gamma_S; \text{pk}_S, (B'_{S,[i]}_{i=0}^3, \hat{A}_S))$   $\triangleright$  *one folded check under  $\text{pk}_S$ : the four new-balance limbs and the aggregate sender-side debit are consistent encryptions*
- 5: **for**  $k = 1$  **to**  $K$  **do**
- 6:     **assert**  $\text{Verify}_{\text{EG}}^{\otimes}(\gamma_{R_k}; \text{pk}_{R_k}, (A_{R_k,[i]}_{i=0}^3))$   $\triangleright$  *one folded check per receiver key*
- 7: **end for**  
 $\triangleright$  *Auditor consistency*
- 8: **if** no auditor data is attached: **assert**  $\text{pk}_{\text{aud}} = \perp$ , and skip to the balance relation  $\triangleright$  *a transfer must carry auditor data whenever a current key is installed; with the current key cleared, data under a still-set outgoing key remains optional*
- 9:  $\tilde{C}_{k,j} \leftarrow A_{R_k,[2j]}.C + 2^{16} A_{R_k,[2j+1]}.C$ ;  $\tilde{E}_{k,j} \leftarrow (\tilde{C}_{k,j}, \tilde{D}_{k,j})$  for all  $k, j$   $\triangleright$  *chunk commitments recomputed homomorphically on-chain; only the  $2K$  handles were posted*
- 10: **assert**  $\text{Verify}_{\text{EG}}^{\otimes}(\gamma_{\text{aud}}; \text{pk}_{\text{aud}}, (\tilde{E}_{k,j})_{k,j})$ , or  $\text{pk}_{\text{aud}}^{\text{old}} \neq \perp$  and  $\text{Verify}_{\text{EG}}^{\otimes}(\gamma_{\text{aud}}; \text{pk}_{\text{aud}}^{\text{old}}, (\tilde{E}_{k,j})_{k,j})$ ; let  $\text{pk}_{\star}$  be the key that verified  $\triangleright$  *the transaction designates no auditor key, so the ledger tries the current key and then the outgoing one, at most two verifications. Every handle is then consistent with its chunk commitment under  $\text{pk}_{\star}$ , so the auditor can decrypt every  $a_k$*   
 $\triangleright$  *Balance relation*
- 11:  $\Delta \leftarrow \text{Collapse}(B'_S) + \hat{A}_S - \text{Collapse}(B_S)$
- 12: **assert**  $\text{Verify}_{\text{DDH}}(\pi_{\text{bal}}; G, \Delta.C, \text{pk}_S, \Delta.D)$   $\triangleright$   $\Delta$  encrypts zero under  $\text{pk}_S$ : what leaves the sender equals the total credited
- 13:  $B_S \leftarrow B'_S$ ;  $\tau_{\text{act}}^S \leftarrow 1$   $\triangleright$   $B'_S$  is freshly range-proven, so the sender's bound counter resets
- 14: **for**  $k = 1$  **to**  $K$  **do**
- 15:      $B_{\text{pnd}}^{R_k} \leftarrow B_{\text{pnd}}^{R_k} + A_{R_k}$ ;  $\tau_{\text{pnd}}^{R_k} += 1$   $\triangleright$  *limb-wise homomorphic add into  $R_k$ 's pending accumulator*
- 16:     **emit**  $\text{TransferEvent}(S, R_k, A_{R_k}, (\tilde{D}_{k,j})_{j \in \{0,1\}}, \text{pk}_{\star})$   $\triangleright$  *the handles and the key that verified them are carried in the event; the ephemeral  $P$  is recorded once per transfer as before*
- 17: **end for**

---

---

**Algorithm 6** Private-to-public conversion (unwrap)

**Require:** Holder keys  $(\text{sk}_U, \text{pk}_U)$ ; on-chain active balance  $B_U$ ; public withdrawal  $v \in \text{u64}$ ,  $v \leq \text{Dec}(\text{sk}_U, B_U)$ .

**Holder (off-chain)**

- 1:  $b_U \leftarrow \text{Dec}(\text{sk}_U, B_U)$ ;  $b'_U \leftarrow b_U - v$ ;  $B'_U \leftarrow \text{EA}(\text{pk}_U, b'_U)$
- 2:  $\Delta \leftarrow \text{Collapse}(B'_U) + \text{TrivEnc}(v) - \text{Collapse}(B_U)$
- 3:  $\pi_{\text{bal}} \leftarrow \Pi_{\text{DDH}}(\text{sk}_U; G, \Delta.C, \text{pk}_U, \Delta.D) \triangleright \text{ensures } \Delta \text{ encrypts zero: the public withdrawal } v \text{ exactly matches the drop in the encrypted balance}$
- 4:  $\mathcal{L}_{\text{rng}} \leftarrow (B'_{U,[0]}.C, \dots, B'_{U,[3]}.C)$
- 5:  $\pi_{\text{range}}^{16} \leftarrow \Pi_{\text{range}}^{16}(\text{openings for } \mathcal{L}_{\text{rng}}; \mathcal{L}_{\text{rng}}) \triangleright \text{ensures the residual balance limbs are non-negative, so the holder cannot withdraw more than it holds}$
- 6:  $\gamma_U \leftarrow \Pi_{\text{EG}}^{\otimes}((v'_i, \rho_{U,i})_{i=0}^3; \text{pk}_U, (B'_{U,[i]})_{i=0}^3) \triangleright \text{one witness-folded proof: every residual-balance limb is well-formed under } \text{pk}_U, \text{ keeping the balance decryptable}$
- 7: **submit**  $(B'_U, v, \pi_{\text{range}}^{16}, \gamma_U, \pi_{\text{bal}})$

**Ledger (on-chain)**

- 8: **assert**  $\text{Verify}_{\text{range}}^{16}(\pi_{\text{range}}^{16}; \mathcal{L}_{\text{rng}})$
  - 9: **assert**  $\text{Verify}_{\text{EG}}^{\otimes}(\gamma_U; \text{pk}_U, (B'_{U,[i]})_{i=0}^3) \triangleright \text{one folded check for all four residual-balance limbs}$
  - 10:  $\Delta \leftarrow \text{Collapse}(B'_U) + \text{TrivEnc}(v) - \text{Collapse}(B_U)$
  - 11: **assert**  $\text{Verify}_{\text{DDH}}(\pi_{\text{bal}}; G, \Delta.C, \text{pk}_U, \Delta.D) \triangleright \Delta \text{ encrypts zero under } \text{pk}_U: \text{the public withdrawal } v \text{ matches the balance drop}$
  - 12:  $B_U \leftarrow B'_U$ ;  $\tau_{\text{act}} \leftarrow 1$ ;  $\text{pool} \leftarrow \text{pool} - v$ ; **return** public coin of value  $v$  to  $U$
-

---

**Algorithm 7** Merge pending deposits; update balance (two independent operations)

---

**Require:** Account state: active balance  $B_U$  with bound counter  $\tau_{\text{act}}$ , pending encrypted  $B_{\text{pnd}}^U$  (with  $\tau_{\text{pnd}}$ ), pending public  $b_{\text{pub}}^U$ ; holder secret  $\text{sk}_U$ .

- ▷ *MERGE (ledger): an independent operation that folds pending into active*
- 1:  $B_U \leftarrow B_U + B_{\text{pnd}}^U + \text{TrivEnc}_{\text{limbs}}(b_{\text{pub}}^U)$  ▷ *homomorphic add pending encrypted + trivial limb encoding of pending public*
- 2:  $\tau_{\text{act}} \leftarrow \tau_{\text{act}} + \tau_{\text{pnd}} + \mathbf{1}[b_{\text{pub}}^U > 0]$  ▷ *see Section 3.1 for the  $\tau < 2^{16}$  invariant; the public accumulator consumes a slot only when nonzero*
- 3:  $B_{\text{pnd}}^U \leftarrow \mathbf{0}, \tau_{\text{pnd}} \leftarrow 0$  ▷ *reset pending encrypted; zero four-limb ciphertext*
- 4:  $b_{\text{pub}}^U \leftarrow 0$

▷ *UPDATE BALANCE (ledger + holder): an independent operation, callable on its own*  
 ▷ *it reads and rewrites only  $B_U$ ; it does not require  $\tau_{\text{pnd}} = 0$  and never touches pending*

**Holder (off-chain)**

- 5:  $b_U \leftarrow \text{Dec}(\text{sk}_U, B_U); B'_U \leftarrow \text{EA}(\text{pk}_U, b_U)$  ▷ *fresh limbs under  $\text{pk}_U$*
- 6:  $\Delta \leftarrow \text{Collapse}(B'_U) - \text{Collapse}(B_U)$
- 7:  $\pi_{\text{eq}} \leftarrow \Pi_{\text{DDH}}(\text{sk}_U; G, \Delta.C, \text{pk}_U, \Delta.D)$  ▷ *ensures  $\Delta$  encrypts zero: the freshly re-encrypted balance holds the same value as the merged one*
- 8:  $\mathcal{L}_{\text{rng}} \leftarrow (B'_{U,[0]}.C, \dots, B'_{U,[3]}.C)$
- 9:  $\pi_{\text{range}}^{16} \leftarrow \Pi_{\text{range}}^{16}(\text{openings for } \mathcal{L}_{\text{rng}}; \mathcal{L}_{\text{rng}})$  ▷ *ensures the re-encoded limbs are back in  $[0, 2^{16})$ , restoring the limb bound after homomorphic accumulation*
- 10:  $\gamma_U \leftarrow \Pi_{\text{EG}}^{\otimes}((v'_i, \rho_{U,i})_{i=0}^3; \text{pk}_U, (B'_{U,[i]}_{i=0}^3))$  ▷ *one witness-folded proof: every re-encoded limb is well-formed under  $\text{pk}_U$ , so the refreshed balance stays decryptable*
- 11: **submit**  $(B'_U, \pi_{\text{range}}^{16}, \gamma_U, \pi_{\text{eq}})$

**Ledger (on-chain)**

- 12: **assert**  $\text{Verify}_{\text{range}}^{16}(\pi_{\text{range}}^{16}; \mathcal{L}_{\text{rng}})$
  - 13: **assert**  $\text{Verify}_{\text{EG}}^{\otimes}(\gamma_U; \text{pk}_U, (B'_{U,[i]}_{i=0}^3))$  ▷ *one folded check for all four re-encoded limbs*
  - 14:  $\Delta \leftarrow \text{Collapse}(B'_U) - \text{Collapse}(B_U)$
  - 15: **assert**  $\text{Verify}_{\text{DDH}}(\pi_{\text{eq}}; G, \Delta.C, \text{pk}_U, \Delta.D)$  ▷  *$\Delta$  encrypts zero under  $\text{pk}_U$ : the refreshed balance equals the merged one*
  - 16:  $B_U \leftarrow B'_U; \tau_{\text{act}} \leftarrow 1$
-

---

**Algorithm 8** Token key rotation

---

**Require:** Current key  $(\mathbf{sk}_U, \mathbf{pk}_U)$  of the account's token  $T$  with  $\mathbf{pk}_U \neq \mathcal{O}$ ; fresh key  $(\mathbf{sk}'_U, \mathbf{pk}'_U)$  with  $\mathbf{pk}'_U = \mathbf{sk}'_U G$ ; active balance  $B_U$  with limbs  $B_{U,[i]} = \text{Enc}(\mathbf{pk}_U, v_i; \rho_{U,i})$  and merge counter  $\tau_{\text{act}}$  (carried over unchanged); no pending *encrypted* deposits ( $\tau_{\text{pnd}} = 0$ ). The public accumulator  $b_{\text{pub}}^U$  need not be zero.

**Holder (off-chain)**

- 1:  $w \leftarrow \mathbf{sk}'_U \mathbf{sk}_U^{-1}$
- 2: **for**  $i = 0$  **to** 3 **do**
- 3:    $D'_i \leftarrow w B_{U,[i]}.D$   $\triangleright$  reuse the commitment  $B_{U,[i]}.C$  and re-key only the handle; keys are per token, so a rotation re-keys only  $T$ 's balance
- 4: **end for**
- 5:  $\pi_{\text{rekey}} \leftarrow \Pi_{\text{rekey}}(w; \mathbf{pk}_U, \mathbf{pk}'_U, (B_{U,[i]}.D, D'_i)_{i=0}^3)$   $\triangleright$  one batched DDH: the quotient  $w$  maps  $\mathbf{pk}_U \mapsto \mathbf{pk}'_U$  and every old handle to its re-keyed counterpart
- 6: **submit**  $(\mathbf{pk}'_U, (D'_i)_{i=0}^3, \pi_{\text{rekey}})$   $\triangleright$  only the re-keyed handles posted; the ledger already holds the reused commitments  $B_{U,[i]}.C$

**Ledger (on-chain)**

- 7: **assert**  $\mathbf{pk}'_U \neq \mathcal{O}$
- 8: **assert**  $\tau_{\text{pnd}} = 0$   $\triangleright$  no pending encrypted deposits.
- 9: **assert**  $\text{Verify}_{\text{rekey}}(\pi_{\text{rekey}}; \mathbf{pk}_U, \mathbf{pk}'_U, (B_{U,[i]}.D, D'_i)_{i=0}^3)$   $\triangleright$  each new handle is the old one transported to  $\mathbf{pk}'_U$  under one shared witness
- 10: **for**  $i = 0$  **to** 3 **do**
- 11:    $B_{U,[i]} \leftarrow (B_{U,[i]}.C, D'_i)$   $\triangleright$  reuse the on-chain commitment unchanged, so by binding it still encodes the same  $v_i$  (whatever its magnitude under the current  $\tau_{\text{act}}$ ); only the handle is swapped, so no fresh range/EG proof is needed
- 12: **end for**
- 13:  $\mathbf{pk}_U \leftarrow \mathbf{pk}'_U$

**Ensure:** An account holding  $m$  tokens rotates each of them in a separate call. A soft-failing variant leaves the token unchanged instead of aborting when  $\pi_{\text{rekey}}$  does not verify, e.g. because a deposit raced the holder's read of  $B_U$ .

---

## 6 Universal-Composability Security

We give a single security statement in the universal-composability (UC) framework [Can01]: the protocol  $\Pi_{\text{ConfTx}}$  of Section 5 UC-realizes an ideal functionality  $\mathcal{F}_{\text{ConfTx}}$  (Figure 2) that maintains confidential state in plaintext and exposes a small, explicit leakage profile to the simulator  $\mathcal{S}$ . Two properties follow directly from the functionality.

- *Integrity.* Every accepted transition is value-preserving by construction, so no environment can drive  $\mathcal{F}_{\text{ConfTx}}$  into inflation, over-spend, or a negative balance.
- *Privacy.*  $\mathcal{S}$  sees only the leakage column of Figure 2: transfer amounts between honest parties are hidden, and only the public boundary crossings (wrap, unwrap) together with the visible transaction graph are revealed.

Both  $\Pi_{\text{ConfTx}}$  and  $\mathcal{F}_{\text{ConfTx}}$  are parameterized by a UC session identifier  $\text{sid}$  (Section 6.1); the realization claim is stated per  $\text{sid}$  (Theorem 3) and lifted to concurrent execution as Corollary 4.

### 6.1 Model

**UC at a glance.** Parties are PPT machines scheduled by a PPT environment  $\mathcal{Z}$ .  $\Pi_{\text{ConfTx}}$  runs in the  $\mathcal{F}_{\text{Ledger}}$ -hybrid model alongside a real adversary  $\mathcal{A}$ ; in the ideal world,  $\mathcal{F}_{\text{ConfTx}}$  runs alongside  $\mathcal{S}$ .  $\Pi$  UC-realizes  $\mathcal{F}$  if for every PPT  $\mathcal{A}$  there exists a PPT  $\mathcal{S}$  such that  $\text{EXEC}_{\Pi, \mathcal{A}, \mathcal{Z}} \stackrel{c}{\approx} \text{EXEC}_{\mathcal{F}, \mathcal{S}, \mathcal{Z}}$  for every PPT  $\mathcal{Z}$ .

**Hybrid functionalities.** The realization uses two hybrid functionalities, both instantiated once per session: the ledger  $\mathcal{F}_{\text{Ledger}}^{\text{sid}}$ , and the Groth–Ostrovsky–Sahai NIZK functionality  $\mathcal{F}_{\text{NIZK}}$  [GOS06], one instance per proof relation, through which all transaction-level NIZKs are produced and verified.

*Ledger.*  $\mathcal{F}_{\text{Ledger}}$  models Sui’s totally-ordered ledger: it accepts well-formed transactions, totally orders them, applies their state effects atomically, and exposes a single immutable public log. Replay protection lives in the ledger, so cryptographic transcripts in  $\Pi$  doesn’t need any per-account version counter. Each session has its own ledger instance  $\mathcal{F}_{\text{Ledger}}^{\text{sid}}$ , holding the state that session reads and writes: the objects of token  $T$  (its confidential-token object and backing pool) and every account’s per-token state for  $T$ . This abstracts the fact that sessions on one chain touch disjoint state.

*Proof systems.* All transaction-level NIZKs run in the  $\mathcal{F}_{\text{NIZK}}$ -hybrid model (Figure 1), the NIZK functionality of Groth, Ostrovsky, and Sahai [GOS06], with one instance  $\mathcal{F}_{\text{NIZK}}^R$  per relation. Its sub-session identifier  $\text{ssid}$  is the per-account Fiat-Shamir label (Appendix A.1). A party proves under its own account’s  $\text{ssid}$ , and the ledger verifies every proof under the  $\text{ssid}$  of the account submitting the transaction. We use two properties of  $\mathcal{F}_{\text{NIZK}}$  in place of standalone zero-knowledge, soundness, and extractability. A proof is produced from the statement alone, so  $\mathcal{S}$  simulates every honest-party proof without a witness. Additionally,  $\mathcal{F}_{\text{NIZK}}$  accepts a proof under a given  $\text{ssid}$  only if it was produced under that  $\text{ssid}$  or a witness has been supplied for it, so  $\mathcal{S}$  obtains a valid witness for every proof the ledger accepts from a corrupt account, with no rewinding.

Well-formedness is guaranteed by combining proofs: the protocol submits several  $\mathcal{F}_{\text{NIZK}}$  instances that share the same Pedersen limb commitments. Here  $\mathcal{F}_{\text{NIZK}}^{\text{EG}}$  is the instance for the witness-folded relation  $\mathcal{R}_{\text{EG}}^{\otimes}$  (Section 2); one proof covers all ciphertexts under one key, and its witness carries one opening  $(m_j, r_j)$  per ciphertext. A four-limb amount is pinned by  $\mathcal{F}_{\text{NIZK}}^{\text{range}}$  (16-bit) and  $\mathcal{F}_{\text{NIZK}}^{\text{EG}}$ ; a transfer’s auditor chunks by that same 16-bit  $\mathcal{F}_{\text{NIZK}}^{\text{range}}$ , inherited through the commitments they are built from, and a second  $\mathcal{F}_{\text{NIZK}}^{\text{EG}}$  instance under  $\text{pk}_{\text{aud}}$ . This is the pattern used throughout: each instance extracts its own opening of a shared commitment, and Pedersen binding under discrete log

**Functionality  $\mathcal{F}_{\text{NIZK}}$**

Parameterized with a relation  $R$  and running with parties  $P_1, \dots, P_n$  and adversary  $\mathcal{S}$ .

- **Proof.** On input  $(\text{prove}, \text{sid}, \text{ssid}, x, w)$  from a party  $P$ , ignore if  $(x, w) \notin R$ . Send  $(\text{prove}, x)$  to  $\mathcal{S}$  and wait for the answer  $\pi$ ; store  $(\text{ssid}, x, \pi)$  and send  $(\text{proof}, \text{sid}, \text{ssid}, \pi)$  to  $P$ .
- **Verification.** On input  $(\text{verify}, \text{sid}, \text{ssid}, x, \pi)$  from a party  $V$ , check whether  $(\text{ssid}, x, \pi)$  is stored. If not, send  $(\text{verify}, \text{ssid}, x, \pi)$  to  $\mathcal{S}$  and wait for the answer  $(\text{witness}, w)$ ; upon receiving it, check whether  $(x, w) \in R$  and, in that case, store  $(\text{ssid}, x, \pi)$ . If  $(\text{ssid}, x, \pi)$  has been stored return  $(\text{verification}, \text{sid}, \text{ssid}, 1)$  to  $V$ , else return  $(\text{verification}, \text{sid}, \text{ssid}, 0)$ .

Figure 1: The NIZK argument functionality  $\mathcal{F}_{\text{NIZK}}$  ([GOS06], Fig. 3), parameterized by the relation  $R$ . The protocol invokes one instance per proof relation  $(\mathcal{R}_{\text{DDH}}, \mathcal{R}_{\text{EG}}^{\otimes}, \mathcal{R}_{\text{range}}^w, \mathcal{R}_{\text{rekey}})$ .

forces the openings to agree, gluing the instances into one witness. That agreement is the only fact the realization needs beyond  $\mathcal{F}_{\text{NIZK}}$ ; with the DDH ciphertext switch it is the whole cryptographic content of the proof, surfacing as the binding aborts  $E_{\text{WEA}}$ ,  $E_{\text{EG}}$ , and  $E_{\text{aud}}$ . Each is a simulator abort raised when the two extracted openings of a shared commitment disagree, breaking Pedersen binding and hence bounded by the discrete-log advantage; we define them precisely at the hops where they arise in the indistinguishability proof below.

*Heuristic instantiation.* A UC NIZK functionality cannot be realized in the plain model, so  $\mathcal{F}_{\text{NIZK}}$  requires a trusted setup: it admits a standard-model realization in the common-reference-string hybrid [GOS06], and is also realizable in variants of the random oracle hybrid [CJS14, LR22]. Our deployment follows the latter route, heuristically instantiating each  $\mathcal{F}_{\text{NIZK}}^R$  by the Fiat–Shamir transform [FS87] of the underlying  $\Sigma$ -protocol  $(\mathcal{R}_{\text{DDH}}, \mathcal{R}_{\text{EG}}^{\otimes}, \mathcal{R}_{\text{rekey}})$  or of Bulletproofs  $(\mathcal{R}_{\text{range}})$ , with a domain-separated hash as the random oracle. Two results partially support this instantiation. Faust et al. [FKMV12] prove the Fiat–Shamir transform of a  $\Sigma$ -protocol with quasi-unique responses simulation-sound and weakly simulation-extractable, and Ganesh et al. [GOP<sup>+</sup>22] prove straight-line simulation-extractability of Fiat–Shamir Bulletproofs in the algebraic group model [FKL18]. These results are model-dependent: the random oracle is used both to derive Fiat–Shamir challenges for soundness and, via programmability, in zero-knowledge simulation, while AGM straight-line extraction only covers algebraic adversaries. They therefore provide evidence for the heuristic instantiation, not a standard-model UC realization of the deployed proofs.

**Session identifiers.** Following the UC convention, both  $\Pi_{\text{ConfTx}}^{\text{sid}}$  and  $\mathcal{F}_{\text{ConfTx}}^{\text{sid}}$  are parameterized by a session identifier  $\text{sid}$  chosen by  $\mathcal{Z}$  at session start. Every input from  $\mathcal{Z}$ , every leakage to  $\mathcal{S}$ , and every message exchanged with  $\mathcal{F}_{\text{Ledger}}^{\text{sid}}$  is tagged with  $\text{sid}$ , and an instance silently drops mismatched tags. Within a single  $\text{sid}$ , the functionality maintains independent per-token and per-account state; distinct  $\text{sids}$  correspond to disjoint protocol instances.

The Fiat-Shamir transcript labels  $\text{sid}, \text{ssid}$  used inside the NIZKs of Appendix A are distinct objects from the UC session identifier; the former is derived deterministically from the latter together with the holder identity and a one-byte protocol tag (Appendix A.1). We overload notation since the surrounding context fixes the intended meaning at every occurrence.

*No sub-session identifier.* Unlike several functionalities, such as UC-(A)PAKE [GMR06, JR18], which add a sub-session identifier  $\text{ssid}$  on top of  $\text{sid}$  to defeat replay across sub-sessions sharing a long-lived random-oracle output (the password hash),  $\Pi_{\text{ConfTx}}^{\text{sid}}$  needs none: each on-chain NIZK statement is freshly randomized per invocation by the holder’s blindings, giving a statement-fresh Fiat-Shamir transcript, and full-transaction replay is ruled out separately by  $\mathcal{F}_{\text{Ledger}}$ . The detailed argument appears in Appendix A.1.

**Corruption.** Corruption is static: at the start of execution,  $\mathcal{Z}$  designates a set  $\mathcal{H}^*$  of corrupt holders and, alongside it, a set  $\mathcal{J}_{\text{cor}}$  of corrupt auditors. The issuer is folded into  $\mathcal{Z}$ , since it holds no per-account decryption key. Corrupt auditors are supported natively here: an auditor’s capability is confined to the transfers whose handles designate its key, so a corrupt auditor leaks exactly the amounts of those transfers, and the leakage rule of Figure 2 states this directly.

## 6.2 The ideal functionality

$\mathcal{F}_{\text{ConfTx}}$  is an ideal functionality that maintains plaintext per-account state for its token. We describe its state, the conventions we follow, and the built-in invariants in the paragraphs below; the per-input rules (validity predicate, plaintext-state update on accept, and message delivered to the simulator  $\mathcal{S}$  as that input’s leakage) are enumerated in Figure 2.

**State.** The state below belongs to a single session  $\mathcal{F}_{\text{ConfTx}}^{\text{sid}}$ ; the *sid* index is implicit on every variable and omitted for legibility. State is disjoint across sessions, since  $\mathcal{F}_{\text{ConfTx}}^{\text{sid}}$  and  $\mathcal{F}_{\text{Ledger}}^{\text{sid}}$  are per-session instances (Section 6.1). The session identifier *sid* fixes a single token type  $T$ , a static parameter of the functionality known up front (like the party set) rather than created at runtime, so  $\mathcal{F}_{\text{ConfTx}}^{\text{sid}}$  governs one token. For this  $T$  the functionality maintains a non-negative pool  $\text{Pool}_T \in \mathbb{Z}_{\geq 0}$  and a single auditor slot  $(\mathcal{A}_T, \mathcal{A}_T^{\text{old}})$  with each entry drawn from the party set together with  $\perp$ , holding the current auditor and the outgoing one, which stays acceptable until the issuer’s next update. Alongside it the functionality keeps an append-only audit log  $\text{Log}_T$  of records  $(S, R, a, \mathcal{A}_\star)$ , one per accepted transfer leg, where  $\mathcal{A}_\star$  is the auditor that transfer *designates*. Records are addressed by their position *idx* in  $\text{Log}_T$ , which the per-transfer **Audit** row of Figure 2 takes as its query and which the real auditor reads off the corresponding **TransferEvent**; the balance row is answered from the tracked **act** and **pnd** instead. For each registered account  $U$  (honest or corrupt) under token  $T$ , it maintains active and pending balances  $\text{act}_U^T, \text{pnd}_U^T \in \mathbb{Z}_{\geq 0}$ , and no auditor state of any kind: auditability is carried by the log, not by the account. Per-account key material lies intentionally outside the functionality: a holder’s public key and any associated proof live on the simulated ledger maintained by  $\mathcal{S}$ . Amount parameters  $a$  and  $v$  are drawn from the u64 input alphabet  $[0, 2^{64})$ .

**Conventions.** Each row of Figure 2 specifies, for one input, the validity predicate, the plaintext-state update on accept, and the *leakage* that  $\mathcal{F}_{\text{ConfTx}}^{\text{sid}}$  delivers to  $\mathcal{S}$  (in Canetti’s syntax [Can01], the message  $\mathcal{F}_{\text{ConfTx}}^{\text{sid}}$  sends to  $\mathcal{S}$  on each accepted input; for the gated rows introduced next, the same message is delivered as the gate query, before acceptance). Leakage tuples in the right column are implicitly *sid*-prefixed; we omit the prefix for brevity. When the table or surrounding prose says  $\mathcal{F}_{\text{ConfTx}}^{\text{sid}}$  *emits* the marker **Failed**, the marker is included in the leakage and  $\mathcal{S}$  realizes it as an entry in the public log of  $\mathcal{F}_{\text{Ledger}}^{\text{sid}}$ , observable to every party of that session. The **AuditUnavailable** marker of the **Audit** row is different: auditing is an off-chain read (Section 3.3), so the marker is returned to the querying auditor alone, enters no leakage, and leaves no public-log entry.

**Availability gate.** The rows of Figure 2 marked *gated* (**Wrap**, **Transfer**, and **Rotate**) are subject to an additional acceptance step. Once the row’s validity predicate (if any) holds,  $\mathcal{F}_{\text{ConfTx}}^{\text{sid}}$  delivers the row’s leakage to  $\mathcal{S}$  and waits for a reply in  $\{\text{ok}, \text{reject}\}$ ; on **ok** it applies the update, while on **reject** it leaves the state unchanged and emits **Failed**. A block of same-sender transfers invoked atomically is gated as a unit: the leakage tuples of the block are delivered together, and the block is discarded with a single **Failed** marker if any pairwise validity predicate fails or  $\mathcal{S}$  replies **reject**. The gate delegates to the adversary the availability of the gated inputs, and only their availability: a **reject** does not mutate state, so the conservation invariant below is unaffected, and the gate query

consists of the leakage the row grants in any case, so  $\mathcal{S}$  receives no additional information. The gate serves to keep the ledger’s public bookkeeping outside the functionality. The real ledger refuses a transfer credit or a wrap that violates the counter invariant of Section 3.1, and refuses a key rotation while pending encrypted deposits exist. Each of these predicates is a deterministic function of the public transaction schedule and is independent of the hidden amounts. The simulator answers a gate query by evaluating the corresponding check on its simulated ledger (*gate accounting*, Section 6.3), so an input is rejected in the ideal world if and only if it is rejected in the real one. For inputs that  $\mathcal{S}$  submits on behalf of corrupt parties the gate is vacuous, as  $\mathcal{S}$  forwards only transactions that its simulated ledger has accepted. Finally, the leakage-only row **Normalize** (the update-balance operation of Algorithm 7) carries no plaintext update; it is included in the interface so that honest update-balance invocations appear on the simulated public log and enter the gate accounting.

**Conservation invariant.** After every accepted input,  $\sum_U(\text{act}_U^T + \text{pnd}_U^T) = \text{Pool}_T$ , which forces  $\text{act}_U^T, \text{pnd}_U^T \leq \text{Pool}_T < 2^{64}$  for every registered  $U$ . The pool bound holds because **Wrap**, the only input that increases  $\text{Pool}_T$ , carries the validity predicate  $\text{Pool}_T + v < 2^{64}$ ; this mirrors the real pool, a u64 **Balance** whose arithmetic aborts on overflow, and both sides of the predicate are determined by the public wrap and unwrap amounts, so it is evaluated identically in the two worlds. The base case is **InitToken**, after which both sides are zero. For the inductive step, each transition either (i) moves  $v$  between  $\text{Pool}_T$  and a single account, (ii) redistributes  $a$  between two registered accounts without touching  $\text{Pool}_T$  (**Transfer** debits  $\text{act}_S^T$  and credits  $\text{pnd}_R^T$ , including the corrupt-corrupt case invoked by  $\mathcal{S}$  on behalf of  $S^*$ ), (iii) rebalances within a single account (**Merge** folds  $\text{pnd}_U^T$  into  $\text{act}_U^T$ ), or (iv) does not touch any value variable (**Register**, **Rotate**, **Normalize**, **UpdateAuditor**, and **Audit**). A gate **reject** leaves the state unchanged, so gated inputs preserve the invariant on both branches. Non-negativity follows from the sender-funds-available preconditions  $a \leq \text{act}_S^T$  and  $v \leq \text{act}_U^T$  on **Transfer** and **Unwrap**, since the value variables are plaintext integers maintained by  $\mathcal{F}_{\text{ConfTx}}$ . The amount range  $[0, 2^{64})$  is an input-alphabet property; encryption and proof-system soundness play no role at this level.

**Leakage classes.** The leakage column partitions inputs into three classes. *Public-by-design* (**InitToken**, **Wrap**, **Unwrap**, **UpdateAuditor**): every parameter, including amounts, is delivered to  $\mathcal{S}$  because these operations cross the public/private boundary or alter publicly observable token configuration. *Identity-only* (**Transfer** between honest parties whose designated auditor is honest, honest **Register**/**Rotate**/**Merge**/**Normalize**): only the participating identifiers and the existence of the operation are revealed; amounts remain hidden. A transfer designating a corrupt auditor,  $\mathcal{A}_\star \in \mathcal{J}_{\text{cor}}$ , is the one downgrade: its amount is leaked, exactly as if an endpoint were corrupt. This is unavoidable and intended, since the real auditor handles are readable with  $\text{sk}_{\mathcal{A}_\star}$ . *Fully private* (both **Audit** rows): nothing is delivered to  $\mathcal{S}$ , but the auditor’s input and output sit between  $\mathcal{Z}$  and  $\mathcal{F}$ , so the returned value is *observable to  $\mathcal{Z}$* :  $\mathcal{Z}$  supplies the query on behalf of an honest  $A$  and reads back either the requested transfer record, the pair  $(\text{act}_U^T, \text{pnd}_U^T)$ , or the **AuditUnavailable** marker. This is what makes the **Audit** rows substantive: matching the two worlds on  $A$ ’s output certifies auditor *correctness* (the designated auditor recovers the true amount of a transfer, and the true balance of a covered account) and auditor *soundness* (a corrupt sender cannot make the recovered value disagree with  $\mathcal{F}$ , i.e. cannot evade audit). The simulator is never notified of audit queries; agreement between the two worlds is established in Section 6.3. Both rows serve honest auditors: a corrupt auditor needs no **Audit** interface, since its information already arrives through the extended **Transfer** leakage. There is no tension with the identity-only **Transfer** row: amount hiding is a guarantee against the network adversary  $\mathcal{S}$ , not against a key-holding auditor, and  $\mathcal{Z}$

acting through an honest  $A$  may read the amounts of the transfers designated to it (and, on a covered account, both endpoints' balances). This disclosure is identical in the two worlds ( $\mathcal{F}$  returns the same values that the real auditor decrypts), so it grants  $\mathcal{Z}$  no distinguishing power.

Corruption modifies the identity-only row: **Transfer** when  $S$  or  $R$  is corrupt forwards the amount, and corrupt-only **Register/Rotate/Merge/Normalize** inputs invoked by  $\mathcal{S}$  on behalf of  $\mathcal{H}^*$  forward the full input tuple. This is unavoidable: a corrupt sender already knows the amount it transfers, a corrupt receiver decrypts its own incoming ciphertext, and corrupt-only operations are initiated by  $\mathcal{A}$  itself.

**Why coverage gates the balance row.** A per-transaction auditor does not hold any account key, so it cannot decrypt  $(\text{act}_U^T, \text{pnd}_U^T)$  but must reconstruct them from the zero balance fixed at registration. Every later change is either public, namely wrap and unwrap amounts and the merges folding them, or a transfer leg, legible exactly when it designates  $A$ . One illegible leg leaves the running total off by an unknown amount that later merges carry forward. Coverage is thus the exact condition for reconstruction from the log alone, and it is decidable from the ledger, since events carry their designated key. The real auditor therefore returns **AuditUnavailable** on exactly the queries  $\mathcal{F}_{\text{ConfTx}}^{\text{sid}}$  rejects, which is what makes the row simulatable; this is discharged at  $G_2 \rightarrow G_3$  in Section 6.3.

**Out-of-scope: privileged issuer paths.** For simplicity,  $\mathcal{F}_{\text{ConfTx}}$  does not model the privileged issuer paths (global pause, per-account freeze and deny list, direct balance overwrite), and Theorem 3 is proven assuming none of them is invoked in the real execution.

### 6.3 Realization

**Theorem 3** (UC realization). *Assume the DDH assumption holds on  $\mathbb{G}$  (which implies hardness of discrete log, hence binding of the Pedersen commitment), and model the transfer key-derivation hash KDF as a random oracle. Then the protocol  $\Pi_{\text{ConfTx}}^{\text{sid}}$  UC-realizes  $\mathcal{F}_{\text{ConfTx}}^{\text{sid}}$  in the  $(\mathcal{F}_{\text{Ledger}}^{\text{sid}}, \mathcal{F}_{\text{NIZK}})$ -hybrid model under static corruption of holders and auditors.*

We fix an arbitrary  $\text{sid}$ , exhibit a per-session simulator  $\mathcal{S} = \mathcal{S}^{\text{sid}}$ , and show that no PPT  $\mathcal{Z}$  distinguishes the real-world execution of  $\Pi_{\text{ConfTx}}^{\text{sid}}$  from the ideal-world execution it induces with more than negligible advantage. References to “the ledger” below mean  $\mathcal{F}_{\text{Ledger}}^{\text{sid}}$ .

**Simulator  $\mathcal{S}$ .**  $\mathcal{S}$  runs  $\mathcal{A}$  internally, emulates  $\mathcal{F}_{\text{Ledger}}^{\text{sid}}$  for it, and reacts to each leakage event of  $\mathcal{F}_{\text{ConfTx}}^{\text{sid}}$  by extending the simulated ledger with a corresponding  $\text{sid}$ -tagged transaction.

*Honest holders.* For each new honest holder  $U$ ,  $\mathcal{S}$  samples a fresh keypair  $(\text{sk}_U^S, \text{pk}_U^S)$ , publishes  $\text{pk}_U^S$ , and maintains all of  $U$ 's transfer-side ciphertexts (active and pending balances, per-receiver amounts, and the aggregate sender-side handle) as encryptions of zero. For a transfer it samples the ephemeral scalar  $e$  itself, publishes  $P = eG$ , and derives the per-limb blindings from  $Z_S = e \text{pk}_U^S$  exactly as an honest sender would, so the simulated blindings follow the real distribution without  $\text{sk}_U^S$ ; the aggregate handle is  $\hat{A}_S.D = r_S \text{pk}_U^S$  for the collapsed blinding  $r_S$  it thereby knows. It additionally posts the auditor handles  $\tilde{D}_{k,j} = \tilde{\rho}_{k,j} \text{pk}_{\text{aud}}$ , computed from the chunk blindings  $\tilde{\rho}_{k,j}$  it already knows; no secret key is needed for these either. Every accompanying proof, including  $\gamma_{\text{aud}}$ , is produced through  $\mathcal{F}_{\text{NIZK}}$ 's proof interface on the statement alone. The sampled  $\text{sk}_U^S$  only fixes the published key and enters no ciphertext, so  $\mathcal{S}$  never uses an honest secret key. For a transfer whose designated auditor is corrupt,  $\mathcal{F}_{\text{ConfTx}}^{\text{sid}}$  leaks  $a$  and  $\mathcal{S}$  instead posts genuine encryptions of the

true limbs: the chunk commitments are shared with the receiver ciphertexts, so an auditor-readable amount forces real receiver ciphertexts too, exactly as in the honest-to-corrupt-receiver case below.

*Corrupt holders.*  $\mathcal{S}$  relays  $\mathcal{A}$ 's ledger submissions verbatim and forwards each accepted transaction to  $\mathcal{F}_{\text{ConfTx}}$ . For an honest-sender **Transfer** to a corrupt  $R^*$ ,  $\mathcal{F}_{\text{ConfTx}}$  leaks the amount and  $\mathcal{S}$  encrypts it under  $\text{pk}_{R^*}$ . For a corrupt-sender **Transfer**,  $\mathcal{S}$  reads the per-receiver witness  $(a_{k,i}, \rho_{k,i})_{i=0}^3$  (the openings of the well-formed encrypted amount  $A_{R_k}$ , each limb  $A_{R_k,[i]} = \text{Enc}(\text{pk}_{R_k}, a_{k,i}; \rho_{k,i})$  with  $a_{k,i} \in [0, 2^{16})$ ) off the  $\mathcal{F}_{\text{NIZK}}^{\text{EG}}/\mathcal{F}_{\text{NIZK}}^{\text{range}}$  interface and sets  $a_k = \sum_i 2^{16i} a_{k,i} \in [0, 2^{64})$ , without using any receiver secret key. With each such transfer  $\mathcal{S}$  additionally forwards the designation  $\mathcal{A}_*$ : the key under which its simulated ledger accepted  $\gamma_{\text{aud}}$ , the current one or, inside the grace window, the outgoing one. This is public data, fixed by the ledger check of Algorithm 5; the witness of  $\gamma_{\text{aud}}$  is not used by  $\mathcal{S}$  at all, and is read only in the hybrid at  $G_2 \rightarrow G_3$  to define the abort  $E_{\text{aud}}$ . A corrupt **Rotate** is accepted only with the re-key checks ( $\text{pk}'_{U^*} \neq \mathcal{O}$ , the reused on-chain commitments  $B_{U^*,[i]}.C$  with only the re-keyed handles  $(D'_i)_{i=0}^3$  posted, and the batched  $\mathcal{F}_{\text{NIZK}}^{\text{rekey}}$  proof binding  $\text{pk}'_{U^*} = w \text{pk}_{U^*}$  and  $D'_i = w B_{U^*,[i]}.D$ ), which preserve the audited balance: commitment reuse fixes the plaintext by binding while the proof only rebases the handles to  $\text{pk}'_{U^*}$ .

*Gate accounting.*  $\mathcal{S}$  answers the availability gates of Figure 2 by evaluating, on its simulated ledger, the same public-bookkeeping checks the real ledger applies. Per account it tracks the bound counters  $\tau_{\text{act}}, \tau_{\text{pnd}}$  and the public accumulator  $b_{\text{pub}}$ , updated on each accepted input exactly as the counter invariant of Section 3.1 prescribes; every update is a deterministic function of the leakage stream.  $\mathcal{S}$  replies ok exactly when the ledger's precondition holds, evaluated in submission order: for each **Transfer** credit to  $R$ , the slot condition  $\tau_{\text{act}}^R + \tau_{\text{pnd}}^R + 2 < 2^{16}$  (Algorithm 5); for a **Wrap** to  $U$ ,  $b_{\text{pub}}^U > 0$  or the same slot condition; for a **Rotate** by  $U$ ,  $\tau_{\text{pnd}}^U = 0$  (Algorithm 8). These quantities are determined by the public transaction schedule, identical in the two worlds, so the gated inputs accepted ideally are exactly those accepted on-chain. Two further predicates fold into the same accounting, both deterministic functions of public state and hence evaluated identically in the two worlds: the auditor designation of the **Transfer** row, fixed by the public auditor slot; and the third-party **Register** gate, which  $\mathcal{S}$  answers by checking on its simulated ledger whether  $U$  has published a default key  $\text{pk}_{\text{dft}}$ .

*Audit.* Both **Audit** rows are answered by  $\mathcal{F}_{\text{ConfTx}}$  directly from its state, the log  $\text{Log}_T$  and the tracked balances, so  $\mathcal{S}$  takes no action; what must hold is that the auditor output read by  $\mathcal{Z}$  agrees across worlds. For a transfer designated to an honest  $A$ , the ideal output is the logged record and the real output is what  $A$  decrypts from the chunk ciphertexts; these agree for honest senders by construction and for corrupt senders by hop  $G_3$  below. The balance row is then the accumulation of those amounts over a covered account. The later switch of honest ciphertexts to zero ( $G_4$ ) is invisible because the audit answer is the tracked plaintext, and it is sound because the obligation exists only for honestly held designated keys, so the corresponding  $\text{sk}_A$  never enters  $\mathcal{Z}$ 's view; a corrupt auditor's view is the adversary's own and is already accounted for in the **Transfer** leakage. Multi-receiver batched transfers reduce to  $K$  pairwise applications of these cases.

**Proof of indistinguishability.** We pass from the real execution  $G_0$  to the ideal execution  $G_5$  in five hops: three witness extractions, one DDH switch, and one information-theoretic step.  $G_0$  is  $\Pi_{\text{ConfTx}}$  in the  $(\mathcal{F}_{\text{Ledger}}, \mathcal{F}_{\text{NIZK}})$ -hybrid model; honest proofs already come from  $\mathcal{F}_{\text{NIZK}}$  on the statement alone, so no separate zero-knowledge hop is needed. The extraction hops read the  $\mathcal{F}_{\text{NIZK}}$  witnesses of accepted corrupt proofs and each adds one binding abort:  $G_1$  the  $\mathcal{F}_{\text{NIZK}}^{\text{EG}}/\mathcal{F}_{\text{NIZK}}^{\text{range}}$  witnesses of corrupt amounts ( $E_{\text{WEA}}$ ),  $G_2$  the opening of the aggregate ciphertext  $\hat{A}_S$  carried in the same folded  $\mathcal{F}_{\text{NIZK}}^{\text{EG}}$  witness ( $E_{\text{EG}}$ ), and  $G_3$  the  $\mathcal{F}_{\text{NIZK}}^{\text{EG}}$  openings of the auditor chunks of each corrupt transfer ( $E_{\text{aud}}$ ). Outside the aborts these hops leave the view unchanged and equip the challenger with every

corrupt plaintext, so the DDH reduction that follows runs without the challenged secret keys and never decrypts: corrupt parties' contributions come from the extracted witnesses.  $G_4$  switches the honest transfer-side ciphertexts to encryptions of zero. One switch suffices, since no ciphertext in the protocol carries a secret key as its plaintext. The final hop  $G_4 \rightarrow G_5$  is information-theoretic: with the extracted openings and the balance relation  $\mathcal{F}_{\text{NIZK}}^{\text{DDH}}$  certifies for every accepted spend, each accepted transition is value-preserving and is forwarded to  $\mathcal{F}_{\text{ConfTx}}$ .

$G_0 \rightarrow G_1$  (*encrypted amounts*). Every proof is verified under the `ssid` of the account submitting it. Read the  $\mathcal{F}_{\text{NIZK}}^{\text{EG}}$  and  $\mathcal{F}_{\text{NIZK}}^{\text{range}}$  witnesses of every accepted corrupt amount: the new balance  $B'_{S^*}$  in transfers, unwraps, balance update, and each receiver amount  $A_{R_k}$ . Add the abort event  $E_{\text{WEA}}$ , raised when the two openings of some shared limb commitment  $C_i$  disagree; this breaks Pedersen binding, so  $\Pr[E_{\text{WEA}}] \leq \text{Adv}_{\mathbb{G}}^{\text{dlog}}$ . On  $\neg E_{\text{WEA}}$  each freshly range-proven limb lies in  $[0, 2^{16})$  (otherwise, an accumulated balance limb is only bounded by  $2^{32}$ ) over  $\mathbb{Z}$  and satisfies  $D_i = r_i \mathbf{pk}$ , so the balance relation lifts to  $\mathbb{Z}$  without wrap-around and the recovered amount  $a_k = \sum_i 2^{16i} m_{k,i}$  lies in  $[0, 2^{64})$ .

$G_1 \rightarrow G_2$  (*aggregate sender consistency*). For every accepted corrupt transfer, read the opening  $(r_{\hat{A}}, m_{\hat{A}})$  of the aggregate ciphertext  $\hat{A}_S$  from the sender-side folded  $\mathcal{F}_{\text{NIZK}}^{\text{EG}}$  witness (the fifth slot of  $\gamma_S$ , alongside the four new-balance limb openings read at  $G_1$ ), with  $\hat{A}_S = (r_{\hat{A}}G + m_{\hat{A}}H, r_{\hat{A}}\mathbf{pk}_{S^*})$ . Add the abort event  $E_{\text{EG}}$ , raised when this opening of  $\hat{A}_S.C$  disagrees with the one forced by the ledger's reconstruction  $\hat{A}_S.C = \sum_k \text{Collapse}(A_{R_k}).C$  and the  $G_1$  openings of the  $A_{R_k}$ ; again this breaks binding, so  $\Pr[E_{\text{EG}}] \leq \text{Adv}_{\mathbb{G}}^{\text{dlog}}$ . On  $\neg(E_{\text{EG}} \cup E_{\text{WEA}})$  the openings agree, fixing  $m_{\hat{A}} = a = \sum_k a_k$  and pinning the aggregate handle to  $\hat{A}_S.D = (\sum_{k,i} 2^{16i} \rho_{k,i}) \mathbf{pk}_S$ .

$G_2 \rightarrow G_3$  (*auditor-chunk consistency*). For every accepted corrupt transfer carrying auditor data, read the  $\mathcal{F}_{\text{NIZK}}^{\text{EG}}$  witness of  $\gamma_{\text{aud}}$ : the openings  $(\tilde{a}_{k,j}, \tilde{\rho}_{k,j})$  of the chunk ciphertexts  $\tilde{E}_{k,j}$  under  $\mathbf{pk}_*$ , the key the ledger accepted, which for a corrupt sender inside the grace window may be the outgoing key rather than  $\mathbf{pk}_{\text{aud}}$ . Add the abort event  $E_{\text{aud}}$ , raised when some opening of  $\tilde{C}_{k,j}$  disagrees with the one forced by the  $G_1$  openings of its two constituent limbs. Since the ledger recomputes  $\tilde{C}_{k,j} = C_{k,2j} + 2^{16}C_{k,2j+1}$  homomorphically, disagreement breaks Pedersen binding, so  $\Pr[E_{\text{aud}}] \leq \text{Adv}_{\mathbb{G}}^{\text{dlog}}$ . On  $\neg E_{\text{aud}}$  each chunk opens to  $\tilde{a}_{k,j} = a_{k,2j} + 2^{16}a_{k,2j+1} < 2^{32}$  over  $\mathbb{Z}$ , the bound inherited from the  $G_1$  range openings, so the honest designated auditor's bounded discrete-log decryption returns exactly the amount recorded in  $\text{Log}_T$ : the per-transfer row  $\text{Audit}(T, \text{tx})$  agrees with the real world on every corrupt sender.

$G_3 \rightarrow G_4$  (*transfer-side switch*). For every honest sender we first replace the KDF-derived transfer blindings by fresh uniform ones, which is invisible to  $\mathcal{A}$ : the ephemeral tuple  $(G, P, \mathbf{pk}_S, Z_S)$  is a DDH instance, so  $Z_S = \text{sk}_S P$  is indistinguishable from a uniform group element, after which  $\mathcal{A}$  queries the random oracle KDF at  $Z_S$  only with probability  $\leq Q/q$ , so the outputs  $\rho_{k,i} = \text{KDF}(Z_S, \cdot)$  are uniform and independent in its view. With fresh blindings the honest transfer matches the pre-derivation distribution, and we replace every honest-key transfer-side ciphertext by an encryption of zero: the new balance  $B'_S$  and, for each honest receiver, the amount  $A_{R_k}$  together with its sender-side handle. Sharing a per-limb commitment, the receiver handle, the sender handle, and a share of a chunk auditor handle form one  $n = 3$  multi-recipient ciphertext under  $(\mathbf{pk}_{R_k}, \mathbf{pk}_S, \mathbf{pk}_{\text{aud}})$  (Section 2); honest  $\text{Rotate}$  is identical. The reduction embeds the DDH challenge per honest key slot, without holding  $\text{sk}_R, \text{sk}_S, \text{sk}_{\text{aud}}$ . The three slots are not symmetric:  $\mathbf{pk}_{R_k}$  and  $\mathbf{pk}_S$  vary per transfer, whereas  $\mathbf{pk}_{\text{aud}}$  is one key shared by every transfer between two auditor updates, so its challenge is embedded once and reused across all of them by random self-reducibility, which the  $\text{poly}(\lambda)$  factor absorbs. Transfers left unswitched still post auditor handles under that same challenged key, which costs nothing, since  $\tilde{D}_{k,j} = \tilde{\rho}_{k,j} \mathbf{pk}_{\text{aud}}$  needs the public key only. The reduction posts

each  $A_{R_k}$  from the receiver-key components, and assembles both linear-combination handles from the per-limb components:  $\hat{A}_S.D$  as the  $2^{16i}$ -weighted sum of the sender-key components, exactly as the baseline already did, and  $\tilde{D}_{k,j}$  as the  $2^{16}$ -weighted pair of the auditor-key components. So  $\hat{A}_S.C = \sum_k \text{Collapse}(A_{R_k}).C$  and  $\tilde{C}_{k,j} = A_{R_k,[2j]}.C + 2^{16}A_{R_k,[2j+1]}.C$  both hold by construction. The switch applies to transfers whose sender, receiver, and designated auditor are all honest. A transfer with  $A_\star \in \mathcal{J}_{\text{cor}}$  leaks its amount and keeps genuine ciphertexts, so it needs no switch, the same treatment the baseline gives honest-to-corrupt-receiver transfers. Corrupt-sender amounts are read from the  $G_1$  witnesses, and honest-to-corrupt amounts stay real encryptions under  $\text{pk}_{R^\star}$ . Accompanying proofs are  $\mathcal{F}_{\text{NIZK}}$  mediated on the statement alone, unaffected by the switch. The hop is bounded by  $\text{poly}(\lambda) \cdot \text{Adv}_{\mathbb{G}}^{\text{DDH}} + Q/q$ , the second term from the random-oracle guess on  $Z_S$ .

$G_4 \rightarrow G_5$  (*ideal world*). The simulator drops its plaintext bookkeeping and defers to  $\mathcal{F}_{\text{ConfTx}}$ , forwarding each accepted corrupt transaction with the amounts read at  $G_1$  and the designation  $A_\star$  its ledger check fixed. Availability gates are answered by the gate accounting of the simulator description; since the gate predicates, the designation predicate among them, are deterministic functions of the public transaction schedule, identical across the two games, the gated accepts and Failed markers of  $G_5$  coincide with the ledger's accepts and rejections in  $G_4$  and contribute no advantage. The only obligation is that every accepted transition preserves value, which the two ledger-checked proofs guarantee:  $\mathcal{F}_{\text{NIZK}}^{\text{DDH}}$  for spends and  $\mathcal{F}_{\text{NIZK}}^{\text{rekey}}$  for key rotation.

- *Spends* ( $\mathcal{F}_{\text{NIZK}}^{\text{DDH}}$ ). Registration and key rotation assert  $\text{pk}_S \neq \mathcal{O}$ , so  $\text{sk}_S \neq 0$ . For  $\Delta = \text{Collapse}(B'_S) + \hat{A}_S - \text{Collapse}(B_S)$  (or its unwrap or balance-update analogue),  $\mathcal{F}_{\text{NIZK}}$  accepts only statements in  $\mathcal{R}_{\text{DDH}}$ , so some  $x$  satisfies  $\text{pk}_S = xG$  and  $\Delta.D = x\Delta.C$ . Since  $G$  generates  $\mathbb{G}$ , the first equation has the single solution  $x = \text{sk}_S$ , hence  $\Delta.D = \text{sk}_S\Delta.C$ . As  $\text{sk}_S \neq 0$  the inverse  $\text{sk}_S^{-1}$  exists, so  $\text{Dec}(\text{sk}_S, \Delta) = \log_H(\Delta.C - \text{sk}_S^{-1}\Delta.D) = \log_H(\Delta.C - \Delta.C) = 0$  and  $\Delta$  encrypts zero. A degenerate  $\text{pk} = \mathcal{O}$  account has  $\text{sk} = 0$ , so  $\text{sk}m_\Delta H = \mathcal{O}$  holds for every  $m_\Delta$ : any  $\Delta = (\alpha G + m_\Delta H, \mathcal{O})$  with  $m_\Delta \neq 0$  then verifies while shifting the balance by  $m_\Delta$ . The  $G_1$  and  $G_2$  openings pin  $\Delta.D$  to the true blindings, the range proofs keep  $b'_S$  and  $a$  in  $[0, 2^{64})$ , and the pre-state  $b_S$  satisfies  $b_S < 2^{64}$  by the conservation invariant ( $\text{act}_U^T + \text{pnd}_U^T \leq \text{Pool}_T < 2^{64}$ ) maintained inductively across accepted transitions rather than by a range proof in this transaction, so this lifts to  $b'_S + a - b_S = 0$  over  $\mathbb{Z}$  without wrap-around.
- *Rotation* ( $\mathcal{F}_{\text{NIZK}}^{\text{rekey}}$ ).  $B'_U$  reuses the commitments of  $B_U$ , so binding fixes the plaintext, and  $\mathcal{F}_{\text{NIZK}}^{\text{rekey}}$  only rebases each handle to  $\text{pk}'_U$ , leaving the balance unchanged.

The balance row  $\text{Audit}(T, U)$  now agrees. On a covered account, every hidden delta of  $U$ 's balance is a designated transfer leg whose amount the auditor recovers as above, a sender's debit being the sum of its batch's legs, and every remaining delta is public: wrap and unwrap amounts appear in clear, the pending-to-active split is tracked through the public Merge entries, and the other operations are value-preserving. For a corrupt Rotate, value preservation rests on  $\mathcal{F}_{\text{NIZK}}^{\text{rekey}}$ . Folding the public log forward from  $U$ 's registration thus reproduces  $(\text{act}_U^T, \text{pnd}_U^T)$  exactly, over  $\mathbb{Z}$  without wrap-around by the conservation invariant. The predicate  $\text{covered}(U, A)$  is itself evaluable in both worlds, since events carry the designated key, so the real auditor returns **AuditUnavailable** exactly when  $\mathcal{F}_{\text{ConfTx}}^{\text{sid}}$  does. The obligation exists only for honestly held designated keys; a corrupt auditor's view is the adversary's own. This hop is the audit-soundness step of the proof.

On  $\neg(E_{\text{WEA}} \cup E_{\text{EG}} \cup E_{\text{aud}})$  each accepted transition then matches its row of Figure 2, and conversely, so  $G_4$  and  $G_5$  are identically distributed.

**Advantage bound.** Only the DDH switch and the three binding aborts contribute; the random-oracle derivation of the transfer blindings adds a negligible guessing term  $Q/q$  once  $Z_S$  is switched,

and zero-knowledge and knowledge soundness are intrinsic to the hybrid. Summing,

$$|\Pr[\text{EXEC}_{\Pi, \mathcal{A}, \mathcal{Z}} = 1] - \Pr[\text{EXEC}_{\mathcal{F}, \mathcal{S}, \mathcal{Z}} = 1]| \leq \text{poly}(\lambda)(\text{Adv}_{\mathbb{G}}^{\text{DDH}} + \text{Adv}_{\mathbb{G}}^{\text{dlog}}) + \text{negl}(\lambda),$$

where  $\text{poly}(\lambda)$  counts the DDH and DL applications and  $\text{negl}(\lambda)$  absorbs the random-oracle guessing terms. Since  $\text{Adv}_{\mathbb{G}}^{\text{dlog}} \leq \text{Adv}_{\mathbb{G}}^{\text{DDH}}$ , the probability is negligible under DDH in the random-oracle model, proving Theorem 3.

**Concurrent composability.** The per-session realization of Theorem 3 lifts to arbitrary concurrent execution by Canetti’s universal-composition theorem.

**Corollary 4** (Concurrent composability). *Under the assumptions of Theorem 3, the multi-session extension of  $\Pi_{\text{ConfTx}}$ , in which arbitrarily many copies  $\Pi_{\text{ConfTx}}^{\text{sid}_1}, \Pi_{\text{ConfTx}}^{\text{sid}_2}, \dots$  (each fixing a token type via its sid) run concurrently, UC-realizes the multi-session extension of  $\mathcal{F}_{\text{ConfTx}}$ .*

## 7 Related Work

The line of work most relevant to us runs from research prototypes to deployed smart-contract systems. *Zether* [BAZB20] was the first homomorphic-ElGamal confidential-payments scheme inside a smart contract (on Ethereum); *Anonymous Zether* [Dia21] adds sender/receiver anonymity. *PGC* [CMTA20] fixed the cryptographic blueprint used by later systems: Twisted ElGamal,  $\Sigma$ -protocol balance proofs, Bulletproofs range proofs [BBB<sup>+</sup>18], and a two-layer auditing capability discussed below. That blueprint was implemented in Solana [Sol24] and adopted by later systems [Ste25, Fou26, CMA26].

**Cryptographic primitives.** Zether encrypts balances with symmetric “ElGamal in the exponent.” PGC introduced *Twisted ElGamal* [CMTA20], in which the ciphertext’s first component is a Pedersen commitment and only the second component (the decryption handle) depends on the recipient’s key; deployed systems adopted that blueprint with Bulletproofs range proofs on the commitment.

**Account model and balance encoding.** Zether and PGC store one ElGamal ciphertext per account, which limits practical balances to what baby-step/giant-step can recover from a single discrete log. Solana and later implementations instead split balances into small limbs, each encrypted separately, so per-limb decryption uses a small precomputed table.

**Race-condition handling.** On a concurrent ledger, a small incoming transfer can arrive after a sender builds a proof but before it executes, and invalidate that proof. Zether avoids this with fixed *epochs*: credits accumulate in a pending balance and roll into the spendable balance only at epoch boundaries, so senders face a deadline tied to epoch length. PGC does not address this race because it keeps a single balance ciphertext and has no pending/active split. Solana keeps separate pending and active (or available) balances and requires an explicit **merge** before spending pending funds.

**Auditing.** Zether provides no built-in auditing. PGC offers two optional layers: zero-knowledge policy proofs on each transfer (*regulation compliance*), and encryption of transfer amounts to a supervisor (*supervision*). Solana follows the supervision style at transfer time.

**Security definitions and proofs.** Zether states *overdraft-safety* (and anonymity for Anonymous Zether) in a game-based model. PGC states game-based soundness for its proofs and encryption. Solana documents [Sol21a, Sol21b] provide game-based security definitions of a confidential payment abstraction.

**Technical contributions of this work.** Relative to recent implementations, we *inherit* the PGC-style core: Twisted ElGamal with Bulletproofs and  $\Sigma$ -protocol proofs, pending/active account state, and a backing pool with public `wrap/unwrap`. What follows is new in our design or formalization relative to that prior work:

1. *Per-transaction auditing at constant proof cost.* PGC’s supervision layer also attaches auditor ciphertexts to each transfer, but pays for them with a separate ciphertext and consistency proof per amount. We attach a transfer’s auditor material, which is  $2K$  decryption handles, one per 32-bit chunk of each receiver amount, whose commitments the ledger recomputes homomorphically from the receiver commitments already on chain (Section 3.3). Because the chunks share the auditor key, a single witness-folded proof  $\Pi_{\text{EG}}^{\otimes}$  covers all of them at  $2G + 2F$  regardless of the number of receivers, and no range proof is needed beyond the 16-bit one the transfer already carries. A grace window on auditor-key rotation keeps in-flight transactions valid, and setting the key to  $\perp$  disables auditing by a single issuer call.
2. *Optimistic merge-then-spend.* Merge-then-spend (Section 3.2, Algorithm 7) can commit a pending fold while the payout is retried, a completion mode the compared deployments do not offer.
3. *Explicit homomorphic-growth accounting.* Recent implementations use chunked limb encodings but do not expose a per-aggregate counter that tracks how much homomorphic addition has widened limb ranges and refuses further pending deposits when slack is exhausted. We attach an integer bound counter  $\tau$  (in units of  $2^{16}$ ) to each encrypted aggregate and package range and ElGamal-consistency checks in the *well-formed encrypted amount* carrier of Definition 1; Sui programmable transactions let a user merge and transfer in one transaction while respecting this bound.
4. *UC security with an explicit leakage profile.* Prior work in this line states game-based overdraft-safety (Zether) or per-proof soundness (PGC). We define an ideal functionality  $\mathcal{F}_{\text{ConfTx}}$  (Section 6) with explicit leakage per operation (including conservation between encrypted balances and the backing pool) and prove UC-realization, yielding compositional guarantees via Canetti’s theorem [Can01].

## 8 Advantages of Twisted ElGamal

The encryption scheme of Section 2 is the *twisted* variant of ElGamal due to Chen et al. [CMTA20], not the classical exponential ElGamal [ElG85] of Zether [BAZB20]. For a key pair  $(\text{sk}, \text{pk})$  with  $\text{pk} = \text{sk}G$ , message  $m$ , and randomness  $r$ ,

$$\begin{aligned}\text{Enc}^{\text{cls}}(\text{pk}, m; r) &= (mG + r\text{pk}, rG), \\ \text{Enc}^{\text{twi}}(\text{pk}, m; r) &= (rG + mH, r\text{pk}).\end{aligned}$$

Both are additively homomorphic and IND-CPA under DDH; the consequential difference is the first component. The classical  $C^{\text{cls}} = mG + r\text{pk}$  commits under bases  $(G, \text{pk})$  whose relation  $\text{pk} = \text{sk}G$  is *known* to the holder, who can therefore re-open it to any value via  $(m+\delta, r-\delta/\text{sk})$ . The twisted  $C^{\text{twi}} = rG + mH$  is a Pedersen commitment [Ped92] under bases  $(G, H)$  of unknown relation:

perfectly hiding, computationally binding, and independent of  $\mathbf{pk}$ . The three protocol features below all rest on these two properties.

**Range proofs apply to the ciphertext directly.** Well-formedness (Definition 1) requires each 16-bit limb to lie in  $[0, 2^{16})$ , certified by a Bulletproof [BBB<sup>+</sup>18] over a list of Pedersen commitments. Under twisted ElGamal each  $E_i.C$  is already such a commitment under the basis  $(G, H)$ , where  $H$  is fixed by hash-to-curve so no party knows its discrete log to  $G$ ; the verifier runs on the limb commitments lifted unchanged from the ciphertext, and a transfer with  $K+1$  amounts batches all  $4(K+1)$  of them into one proof. Classical ElGamal cannot use  $(G, \mathbf{pk})$  as the CRS at all: soundness requires the prover to know no relation among the generators, yet the holder is the prover and knows  $\mathbf{pk} = \mathbf{sk}G$  (the re-opening above is exactly this failure). Zether’s workaround keeps the CRS as  $(G, H)$ , attaches to each amount an auxiliary commitment  $\tilde{C} = mH + sG$ , proves the range on  $\tilde{C}$ , and adds a  $\Sigma$ -proof that  $\tilde{C}$  and  $C^{\text{cls}}$  open to the same  $m$ , costing one extra group element and one equality proof per amount.

**Cross-key amount equality needs no plaintext-equality proof.** A private transfer must account for each amount  $a_k$  under two keys: the receiver’s credit is encrypted under  $\mathbf{pk}_{R_k}$  as  $A_{R_k}$ , while the sender’s aggregate debit is charged under  $\mathbf{pk}_S$ ; see Algorithm 4. Because  $C^{\text{twi}}$  is independent of  $\mathbf{pk}$ , the sender’s debit reuses the receiver blindings, so the two sides share the same commitments and differ only in their decryption handles ( $r\mathbf{pk}_S$  versus  $r\mathbf{pk}_{R_k}$ ). The sender therefore posts no separate debit ciphertext: the ledger reconstructs the aggregate debit commitment  $\hat{A}_S.C = \sum_k \text{Collapse}(A_{R_k}).C$  from the on-chain receiver commitments and pairs it with a single aggregate handle  $\hat{A}_S.D = r_S\mathbf{pk}_S$ , tied to  $\mathbf{pk}_S$  by one  $\Pi_{\text{EG}}$  proof and needing no plaintext-equality proof. Classical ElGamal cannot share commitments:  $C^{\text{cls}} = mG + r\mathbf{pk}$  depends on  $\mathbf{pk}$ , so the same amount under two keys differs in both components and a full  $\Sigma$ -protocol of plaintext equality is required.

**Auditor handles do not need full ciphertexts.** Per-transaction auditing (Section 3.3) attaches to each transfer leg two auditor handles  $\tilde{D}_{k,j} = \tilde{\rho}_{k,j}\mathbf{pk}_{\text{aud}}$ , whose chunk commitments the ledger recomputes as fixed linear combinations  $\tilde{C}_{k,j} = A_{R_k,[2j]}.C + 2^{16}A_{R_k,[2j+1]}.C$  of receiver commitments already on chain. Both properties of  $C^{\text{twi}}$  are in play: independence of  $\mathbf{pk}$  lets one commitment serve the receiver, the sender’s aggregate debit, and the auditor at once, and additive homomorphism lets the pair of 16-bit limbs be read as a single 32-bit chunk. An auditor’s copy of an amount therefore costs one group element per chunk, without any additional commitment, and one folded  $\Pi_{\text{EG}}^{\otimes}$  under  $\mathbf{pk}_{\text{aud}}$  binds every chunk of the batch. Classical ElGamal admits no such sharing:  $C^{\text{cls}} = mG + r\mathbf{pk}$  depends on the recipient key, so an auditor copy is a fresh ciphertext committed under the prover-known bases  $(G, \mathbf{pk}_{\text{aud}})$ , which, as in the range-proof case, does not admit a direct certificate and forces an auxiliary commitment and an equality proof per amount.

| Functionality $\mathcal{F}_{\text{ConfTx}}^{\text{sid}}$                                          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |                                                                                                                                                                    |
|---------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Input                                                                                             | Validity ( <i>italic</i> ) and update on accept                                                                                                                                                                                                                                                                                                                                                                                                                                                        | Leakage to $\mathcal{S}$                                                                                                                                           |
| InitToken( $T$ ) <i>from issuer</i>                                                               | $T$ unset: $\text{Pool}_T \leftarrow 0$ , $\mathcal{A}_T, \mathcal{A}_T^{\text{old}} \leftarrow \perp$ ,<br>$\text{Log}_T \leftarrow \varepsilon$                                                                                                                                                                                                                                                                                                                                                      | ( $T$ )                                                                                                                                                            |
| UpdateAuditor( $T, \mathcal{A}', \mathcal{A}^{\text{old}'}$ )<br><i>from issuer</i>               | $\mathcal{A}_T \leftarrow \mathcal{A}'$ ; $\mathcal{A}_T^{\text{old}} \leftarrow \mathcal{A}^{\text{old}'}$ (either possibly $\perp$ ; normally $\mathcal{A}^{\text{old}'}$ is the key being replaced). May be invoked atomically with InitToken.                                                                                                                                                                                                                                                      | ( $T, \mathcal{A}', \mathcal{A}^{\text{old}'}$ )                                                                                                                   |
| Register( $T$ ) <i>from <math>U</math>, or from any <math>P</math> on <math>U</math>'s behalf</i> | $U$ unregistered; gated when $P \neq U$ :<br>$\text{act}_U^T, \text{pnd}_U^T \leftarrow 0$ . On reject: emit Failed.                                                                                                                                                                                                                                                                                                                                                                                   | ( $U, T, P$ )<br>Failed if rejected                                                                                                                                |
| Rotate( $T$ )<br><i>from <math>U</math></i>                                                       | Gated: new per-token key incarnation, preserving $\text{act}_U^T$ and $\text{pnd}_U^T$ . On reject: emit Failed.                                                                                                                                                                                                                                                                                                                                                                                       | ( $U, T$ )<br>Failed if rejected                                                                                                                                   |
| Wrap( $T, U, v$ )<br><i>from payer <math>P</math> and a public coin of value <math>v</math></i>   | $\text{Pool}_T + v < 2^{64}$ , gated. On accept:<br>$\text{Pool}_T += v$ ; $\text{pnd}_U^T += v$ , return Wrapped.<br>Else return Failed                                                                                                                                                                                                                                                                                                                                                               | ( $P, U, v$ )<br>Failed if rejected                                                                                                                                |
| Transfer( $T, R, a, \mathcal{A}_\star$ )<br><i>from <math>S</math></i>                            | $a \leq \text{act}_S^T$ ; $\mathcal{A}_\star = \mathcal{A}_T$ , or $\mathcal{A}_\star = \mathcal{A}_T^{\text{old}} \neq \perp$ ,<br>or $\mathcal{A}_\star = \perp = \mathcal{A}_T$ ; gated. On accept:<br>$\text{act}_S^T -= a$ ; $\text{pnd}_R^T += a$ ; append<br>( $S, R, a, \mathcal{A}_\star$ ) to $\text{Log}_T$ . Honest senders always designate $\mathcal{A}_\star = \mathcal{A}_T$ . Else reject; emit Failed. Several same-sender transfers can be invoked atomically (gated as one block). | ( $S, R, \mathcal{A}_\star$ ); $a$ if<br>$\{S, R\} \cap \mathcal{H}^* \neq \emptyset$<br>or $\mathcal{A}_\star \in \mathcal{J}_{\text{cor}}$<br>Failed if rejected |
| Unwrap( $T, v$ ) <i>from <math>U</math></i>                                                       | $v \leq \text{act}_U^T$ . On accept: $\text{Pool}_T -= v$ ;<br>$\text{act}_U^T -= v$ ; return Unwrapped( $v$ ). Else return Failed                                                                                                                                                                                                                                                                                                                                                                     | ( $U, v$ )                                                                                                                                                         |
| Merge( $T$ ) <i>from <math>U</math></i>                                                           | $\text{act}_U^T += \text{pnd}_U^T$ ; $\text{pnd}_U^T \leftarrow 0$ .                                                                                                                                                                                                                                                                                                                                                                                                                                   | ( $U$ )                                                                                                                                                            |
| Normalize( $T$ ) <i>from <math>U</math></i>                                                       | No plaintext update.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | ( $U$ )                                                                                                                                                            |
| Audit( $T, \text{idx}$ )<br><i>from <math>A</math></i>                                            | $\text{Log}_T[\text{idx}] = (S, R, a, \mathcal{A}_\star)$ exists with<br>$\mathcal{A}_\star = A$ : return ( $S, R, a$ ) to $A$ . Else reject; return AuditUnavailable.                                                                                                                                                                                                                                                                                                                                 | none                                                                                                                                                               |
| Audit( $T, U$ )<br><i>from <math>A</math></i>                                                     | covered( $U, A$ ): every record of $\text{Log}_T$<br>touching $U$ , as sender or receiver, since<br>$U$ 's registration has designation $A$ : return<br>( $\text{act}_U^T, \text{pnd}_U^T$ ) to $A$ . Else reject; return<br>AuditUnavailable.                                                                                                                                                                                                                                                         | none                                                                                                                                                               |

Figure 2: Per-input interface of  $\mathcal{F}_{\text{ConfTx}}^{\text{sid}}$ : validity predicate, plaintext-state update on accept, and leakage delivered to the simulator. Inputs and leakage tuples are implicitly sid-tagged; state, generic interface loop, and built-in invariants are described in the prose preceding this figure.

## References

- [AFK22] Thomas Attema, Serge Fehr, and Michael Klooß. Fiat-Shamir transformation of multi-round interactive proofs. In Eike Kiltz and Vinod Vaikuntanathan, editors, *TCC 2022, Part I*, volume 13747 of *LNCS*, pages 113–142. Springer, Cham, November 2022.
- [BAZB20] Benedikt Bünz, Shashank Agrawal, Mahdi Zamani, and Dan Boneh. Zether: Towards privacy in a smart contract world. In Joseph Bonneau and Nadia Heninger, editors, *FC 2020*, volume 12059 of *LNCS*, pages 423–443. Springer, Cham, February 2020.
- [BBB<sup>+</sup>18] Benedikt Bünz, Jonathan Bootle, Dan Boneh, Andrew Poelstra, Pieter Wuille, and Greg Maxwell. Bulletproofs: Short proofs for confidential transactions and more. In *2018 IEEE Symposium on Security and Privacy*, pages 315–334. IEEE Computer Society Press, May 2018.
- [Can01] Ran Canetti. Universally composable security: A new paradigm for cryptographic protocols. In *42nd FOCS*, pages 136–145. IEEE Computer Society Press, October 2001.
- [CJS14] Ran Canetti, Abhishek Jain, and Alessandra Scafuro. Practical UC security with a global random oracle. In Gail-Joon Ahn, Moti Yung, and Ninghui Li, editors, *ACM CCS 2014*, pages 597–608. ACM Press, November 2014.
- [CMA26] Murat Cenk, Aanchal Malhotra, and Joseph A. Akinyele. Confidential transfers for multi-purpose tokens on the XRP ledger. Cryptology ePrint Archive, Paper 2026/602, 2026.
- [CMTA20] Yu Chen, Xuecheng Ma, Cong Tang, and Man Ho Au. PGC: Decentralized confidential payment system with auditability. In Liqun Chen, Ninghui Li, Kaitai Liang, and Steve A. Schneider, editors, *ESORICS 2020, Part I*, volume 12308 of *LNCS*, pages 591–610. Springer, Cham, September 2020.
- [CP93] David Chaum and Torben P. Pedersen. Wallet databases with observers. In Ernest F. Brickell, editor, *CRYPTO’92*, volume 740 of *LNCS*, pages 89–105. Springer, Berlin, Heidelberg, August 1993.
- [Dia21] Benjamin E. Diamond. Many-out-of-many proofs and applications to anonymous zether. In *2021 IEEE Symposium on Security and Privacy*, pages 1800–1817. IEEE Computer Society Press, May 2021.
- [dVGH<sup>+</sup>23] Henry de Valence, Jack Grigg, Mike Hamburg, Isis Lovecruft, George Tankersley, and Filippo Valsorda. The ristretto255 and decaf448 groups. RFC 9496, IETF, 2023.
- [ElG85] Taher ElGamal. A public key cryptosystem and a signature scheme based on discrete logarithms. *IEEE Transactions on Information Theory*, 31(4):469–472, 1985.
- [FKL18] Georg Fuchsbauer, Eike Kiltz, and Julian Loss. The algebraic group model and its applications. In Hovav Shacham and Alexandra Boldyreva, editors, *CRYPTO 2018, Part II*, volume 10992 of *LNCS*, pages 33–62. Springer, Cham, August 2018.
- [FKMV12] Sebastian Faust, Markulf Kohlweiss, Giorgia Azzurra Marson, and Daniele Venturi. On the non-malleability of the Fiat-Shamir transform. In Steven D. Galbraith and Mridul Nandi, editors, *INDOCRYPT 2012*, volume 7668 of *LNCS*, pages 60–79. Springer, Berlin, Heidelberg, December 2012.

- [Fou26] Aptos Foundation. AIP-143: Confidential APT. Aptos Improvement Proposal, 2026. Status: Accepted.
- [FS87] Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In Andrew M. Odlyzko, editor, *CRYPTO’86*, volume 263 of *LNCS*, pages 186–194. Springer, Berlin, Heidelberg, August 1987.
- [GMR06] Craig Gentry, Philip MacKenzie, and Zulfikar Ramzan. A method for making password-based key exchange resilient to server compromise. In Cynthia Dwork, editor, *CRYPTO 2006*, volume 4117 of *LNCS*, pages 142–159. Springer, Berlin, Heidelberg, August 2006.
- [GOP<sup>+</sup>22] Chaya Ganesh, Claudio Orlandi, Mahak Pancholi, Akira Takahashi, and Daniel Tschudi. Fiat-Shamir bulletproofs are non-malleable (in the algebraic group model). In Orr Dunkelman and Stefan Dziembowski, editors, *EUROCRYPT 2022, Part II*, volume 13276 of *LNCS*, pages 397–426. Springer, Cham, May / June 2022.
- [GOS06] Jens Groth, Rafail Ostrovsky, and Amit Sahai. Perfect non-interactive zero knowledge for NP. In Serge Vaudenay, editor, *EUROCRYPT 2006*, volume 4004 of *LNCS*, pages 339–358. Springer, Berlin, Heidelberg, May / June 2006.
- [JR18] Charanjit S. Jutla and Arnab Roy. Smooth NIZK arguments. In Amos Beimel and Stefan Dziembowski, editors, *TCC 2018, Part I*, volume 11239 of *LNCS*, pages 235–262. Springer, Cham, November 2018.
- [LR22] Anna Lysyanskaya and Leah Namisa Rosenbloom. Universally composable  $\Sigma$ -protocols in the global random-oracle model. In Eike Kiltz and Vinod Vaikuntanathan, editors, *TCC 2022, Part I*, volume 13747 of *LNCS*, pages 203–233. Springer, Cham, November 2022.
- [Ped92] Torben P. Pedersen. Non-interactive and information-theoretic secure verifiable secret sharing. In Joan Feigenbaum, editor, *CRYPTO’91*, volume 576 of *LNCS*, pages 129–140. Springer, Berlin, Heidelberg, August 1992.
- [Sol21a] Solana Labs. Part 1: SPL ZK-Tokens protocol overview. Solana ZK-Token protocol paper, December 2021.
- [Sol21b] Solana Labs. Part 2: SPL ZK-Tokens proof of security. Solana ZK-Token protocol paper, December 2021.
- [Sol24] Solana Foundation. Confidential Transfer extension on Token-2022. Solana developer documentation, 2024.
- [Ste25] Stellar Development Foundation. stellar-confidential-token: Prototype for confidential tokens on Stellar. GitHub repository, 2025.

## A Zero-Knowledge Proof Protocols

The security analysis (Section 6.1) invokes these proof systems through the  $\mathcal{F}_{\text{NIZK}}$ -hybrid model: each is modeled as an instance of the [GOS06] NIZK functionality for its relation, so the UC

proof draws zero-knowledge and straight-line witness extraction directly from  $\mathcal{F}_{\text{NIZK}}$  rather than from standalone game-based properties. The heuristic realization of each  $\mathcal{F}_{\text{NIZK}}$  instance by the Fiat–Shamir transform, with references to the relevant papers, is discussed in Section 6.1. In this section we establish the underlying standalone constructions and their proof-of-knowledge guarantees, for the four relations of Section 2:

1.  $\Pi_{\text{DDH}}$  is the Chaum–Pedersen [CP93] NIZK proof of knowledge for  $\mathcal{R}_{\text{DDH}}$  (Section A.2, Figure 3);
2.  $\Pi_{\text{EG}}$  is a NIZK proof of knowledge for  $\mathcal{R}_{\text{EG}}$ , obtained by applying the Fiat–Shamir transform to the  $\Sigma$ -protocol  $\Sigma_{\text{EG}}$  (Section A.3, Lemma 1, Figure 4);
3.  $\Pi_{\text{range}}$  is an aggregate range proof of knowledge for  $\mathcal{R}_{\text{range}}^w$  ( $w = 16$ ), instantiated by Bulletproofs [BBB<sup>+</sup>18]; and
4.  $\Pi_{\text{rekey}}$  is a NIZK proof of knowledge for  $\mathcal{R}_{\text{rekey}}$ , a batched Chaum–Pedersen [CP93] discrete-log-equality statement obtained by applying the Fiat–Shamir transform to the  $\Sigma$ -protocol  $\Sigma_{\text{rekey}}$  (Section A.4, Lemma 2, Figure 5).

The well-formedness of an encrypted amount that the ledger checks on-chain (Section 5), and likewise the auditor-chunk check of a transfer, are conjunctions of these base proofs over shared limb commitments; the binding argument gluing the shared openings, and the resulting discrete-log reductions, are carried out in the simulator analysis (Section 6.1). In the algorithms of Section 5, the  $\Pi_{\text{EG}}$  instances sharing one public key are carried as a single witness-folded proof  $\Pi_{\text{EG}}^{\otimes}$  (Appendix B); in particular, the transfer’s sender-side proof  $\gamma_S$  folds the four new-balance limbs together with the aggregate sender-side ciphertext  $\hat{A}_S$ , whose commitment the ledger reconstructs from the receiver commitments. Lemma 3 shows the folded proof is a NIZK proof of knowledge for the conjunction relation, so the per-ciphertext analysis below applies unchanged.

## A.1 Domain Separation

**Goals.** Domain separators serve two purposes. Practically, they prevent a proof produced by one user or protocol from being reused in another. In the analysis, they differentiate the random oracles of different parties, so that a simulator can program the oracle for honest parties and extract inputs from corrupted ones.

A unique domain separator should therefore be associated with the high-level protocol and the user. Subprotocol identifiers are optional, since the simulated execution may use a single ideal ZK functionality, and hence a single random oracle, per party.

For the security of the Fiat–Shamir transform, the inputs to the random oracle should include the instance, the group parameters (if not fixed), additional public information (*aad*), and the first message of the interactive ZK protocol.

**Implementation.** The sigma-protocol NIZKs each use a unique 21-byte session identifier *sid* as an additional input to their Fiat–Shamir hash to ensure domain separation between protocol instances and prevent cross-instance attacks. The Bulletproofs range proofs are bound to the same session identifier format. The session identifier is built in two steps. At account registration the 20-byte session identifier basis is fixed to the first 20 bytes of the user’s per-token account object address,

$$\text{base} = \text{addr}(\text{TokenAccount}\langle T \rangle)[0..20],$$

where  $\text{addr}(\text{TokenAccount}\langle T \rangle)$  is deterministically derived by the ledger from the user’s account object and the token type  $T$ . The generators  $G$  and  $H$  are fixed Ristretto255 constants and are not part of the  $\text{sid}$ ; the implementation instead prepends their encodings to every Fiat–Shamir hash as a defense-in-depth measure (item 4 below). To form the  $\text{sid}$ , this basis is completed by appending a single-byte protocol identifier  $b$  characterizing the subprotocol,  $\text{sid} = \text{base} \parallel b$ , where, for example,  $b = 0x01$  for  $\Pi_{\text{DDH}}$ . The protocol byte is used for tightness of the reduction, not for soundness.

**Necessity of each hash input.** Each Fiat-Shamir challenge has the form  $c = H(\text{sid}, G, H, \text{stmt}, \text{first\_msg})$  with  $\text{sid} = \text{addr}(\text{TokenAccount}\langle T \rangle)[0..20] \parallel b$ . We classify each input by the cost of removing it.

1. *Attack without it.* The statement and first-message components in every NIZK: removing any of them breaks Fiat-Shamir directly, by letting the prover choose the omitted field after seeing  $c$ .
2. *No known attack, but needed for the proof.* The account address is prepended to every NIZK hash, binding each proof to the account that produced it and, through the derivation from the UC session identifier described in Section 6.1, to the session (Corollary 4). Some NIZKs, such as  $\Pi_{\text{range}}^{16}$ , are not bound to a user public key by their statement alone; without the address, a corrupted account could replay an honest account’s proof on the same statement, for which the simulator has no witness to extract, and the per-account simulation-soundness/extractability hops would fail.
3. *Only loosens the reduction.* The protocol byte  $b$  keeps the five NIZKs’ oracle slices disjoint, making the per-NIZK hops of Theorem 3 modular; dropping it costs a union bound over the statements ( $Q^2/2^\lambda$ ,  $Q$  the total honest proofs).
4. *Redundant, but kept as defense-in-depth.* The generators  $G, H$  are fixed Ristretto255 constants whose discrete-log relationship is already pinned by the public parameters, so hashing them strengthens no reduction; the implementation nevertheless prepends their encodings to every Fiat–Shamir hash as defense-in-depth against a deployment that misconfigures the generators. The protocol figures in this paper omit these constant inputs for brevity.

**No per-invocation sub-session label.** The  $\text{sid}$  defined above is per-protocol-and-account, not per-invocation: a holder reuses the same  $\text{sid}$  across every **Transfer**, **Wrap**, **Unwrap**, and **Merge** they produce. This is intentional: the per-account label is itself the  $\text{ssid}$  of  $\mathcal{F}_{\text{NIZK}}$ , and no per-invocation identifier is needed on top of it. Every NIZK hash includes the statement and the  $\Sigma$ -protocol first message, both of which an honest holder freshly randomizes per proof. Two honest invocations under the same  $\text{sid}$  thus hash distinct inputs with overwhelming probability, so the random oracle is queried at distinct points and the statement plays the role of a per-invocation identifier  $\text{ssid}$ . The bound is on honest holders only: corrupt provers may reuse randomness, but their proofs are extracted rather than simulated, and the account address keeps each account’s hash slice disjoint, so reuse cannot collide with a programmed point. Replay of an entire accepted invocation is blocked by the ledger functionality (Section 6.1).

## A.2 DDH

The DDH proof is a zero-knowledge proof-of-knowledge for the relation  $\mathcal{R}_{\text{DDH}}$  defined in Section 2: the prover demonstrates knowledge of a witness  $x \in \mathbb{Z}_q$  satisfying  $X = xE$  and  $Y = xF$  simultaneously. The underlying interactive protocol is the classical three-move  $\Sigma$ -protocol of Chaum

| $\Pi_{\text{DDH}}.\text{Prove}(x; \text{sid}, E, F, X, Y) :$ | $\Pi_{\text{DDH}}.\text{Verify}(\pi; \text{sid}, E, F, X, Y) :$ |
|--------------------------------------------------------------|-----------------------------------------------------------------|
| 1 : $r \leftarrow \mathbb{Z}_q$                              | 1 : $(X', Y', z) = \pi$                                         |
| 2 : $X' = rE, Y' = rF$                                       | 2 : $c = \text{H}(\text{sid}, E, F, X, Y, X', Y')$              |
| 3 : $c = \text{H}(\text{sid}, E, F, X, Y, X', Y')$           | 3 : $b_1 = (zE \stackrel{?}{=} X' + cX)$                        |
| 4 : $z = r + cx$                                             | 4 : $b_2 = (zF \stackrel{?}{=} Y' + cY)$                        |
| 5 : <b>return</b> $\pi = (X', Y', z)$                        | 5 : <b>return</b> $b_1 \wedge b_2$                              |

Figure 3: NIZK DDH Protocol  $\Pi_{\text{DDH}}$ ; proof size:  $2\mathbb{G} + \mathbb{F}$

and Pedersen [CP93], which has perfect completeness, 2-special soundness, and perfect special honest-verifier zero-knowledge. Applying the Fiat–Shamir transformation in the random oracle model yields the non-interactive variant  $\Pi_{\text{DDH}}$  shown in Figure 3, which we use throughout this work.

### A.3 ElGamal PoK

We construct a NIZK PoK for the relation  $\mathcal{R}_{\text{EG}}$  of a single Twisted ElGamal ciphertext  $(C, D)$  under public key  $\text{pk}$ , defined in Section 2, and prove its security. We denote its defining equations  $C = rG + mH$  and  $D = r\text{pk}$  with (C1) and (C2), respectively. We analyze  $\Sigma_{\text{EG}}$ , the public-coin three-move  $\Sigma$ -protocol whose Fiat–Shamir form is shown in Figure 4: the interactive protocol replaces the hash-derived challenge with a verifier-sampled  $c \in \mathbb{Z}_q$ . Applying the Fiat–Shamir (FS) transformation then yields  $\Pi_{\text{EG}} = \text{FS}[\Sigma_{\text{EG}}]$ , a NIZK proof-of-knowledge for  $\mathcal{R}_{\text{EG}}$  in the random oracle model. We refer to the two verifier checks in Figure 4 as (V1) and (V2), respectively.

**Lemma 1.** *The protocol  $\Sigma_{\text{EG}}$  is a public-coin 3-move  $\Sigma$ -protocol for the relation  $\mathcal{R}_{\text{EG}}$ .*

*Proof.* To prove the lemma, we show completeness, 2-special soundness, and perfect special honest-verifier zero-knowledge.

*Perfect Completeness.* For an honest prover, the verification equations (V1) and (V2) hold with probability 1 since:

- $z_1G + z_2H = (r_1 + cr)G + (r_2 + cm)H = r_1G + r_2H + c(rG + mH) = C' + cC$
- $z_1\text{pk} = (r_1 + cr)\text{pk} = r_1\text{pk} + cr\text{pk} = D' + cD$

Thus, completeness holds.

*2-Special Soundness.* Given two accepting transcripts  $(C', D', c, z_1, z_2)$  and  $(C', D', c', z'_1, z'_2)$  with  $c \neq c'$ , we show that there exists a PPT extractor  $\text{Ext}$  that can extract a witness  $w$  satisfying (C1) and (C2). Since both transcripts satisfy (V2), we obtain  $(z_1 - z'_1)\text{pk} = (c - c')D$  by subtraction. Since  $c \neq c'$ , the value  $c - c'$  is invertible in  $\mathbb{Z}_q$  and the scalar  $r = (z_1 - z'_1)/(c - c')$  is well-defined and satisfies  $D = r\text{pk}$ . This establishes (C2). Further, since both transcripts satisfy (V1), we obtain  $(z_1 - z'_1)G + (z_2 - z'_2)H = (c - c')C$  by subtraction. Substituting  $(z_1 - z'_1) = r(c - c')$  and dividing by  $(c - c')$ , we obtain  $rG + ((z_2 - z'_2)/(c - c'))H = C$ , which uniquely determines  $m = (z_2 - z'_2)/(c - c')$  such that  $C = rG + mH$ . This establishes (C1) and overall completes witness extraction, showing that 2-special soundness holds.

| $\Pi_{\text{EG}}.\text{Prove}(m, r; \text{sid}, (C, D), \text{pk}) :$ | $\Pi_{\text{EG}}.\text{Verify}(\pi; \text{sid}, (C, D), \text{pk}) :$ |
|-----------------------------------------------------------------------|-----------------------------------------------------------------------|
| 1 : $r_1, r_2 \leftarrow \mathbb{Z}_q$                                | 1 : $(C', D', z_1, z_2) = \pi$                                        |
| 2 : $C' = r_1 G + r_2 H, D' = r_1 \text{pk}$                          | 2 : $c = \text{H}(\text{sid}, \text{pk}, C, D, C', D')$               |
| 3 : $c = \text{H}(\text{sid}, \text{pk}, C, D, C', D')$               | 3 : $b_1 = (z_1 G + z_2 H \stackrel{?}{=} C' + cC)$                   |
| 4 : $z_1 = r_1 + cr, z_2 = r_2 + cm$                                  | 4 : $b_2 = (z_1 \text{pk} \stackrel{?}{=} D' + cD)$                   |
| 5 : <b>return</b> $\pi = (C', D', z_1, z_2)$                          | 5 : <b>return</b> $b_1 \wedge b_2$                                    |

Figure 4: NIZK PoK Protocol for Well-formedness of Twisted ElGamal Encryption

*Perfect Special Honest-Verifier Zero-Knowledge.* Assuming the verifier is honest, we show that there exists a PPT simulator  $\mathcal{S}$  that, given the statement  $s$  and the challenge  $c$  in advance, produces a transcript  $(C', D', c, z_1, z_2)$  that is identically distributed to that of an honest prover. The simulator starts by sampling values  $c, z_1, z_2 \in \mathbb{Z}_q$  uniformly at random. It then sets  $C' = z_1 G + z_2 H - cC$  and  $D' = z_1 \text{pk} - cD$ . In an honest execution, the responses  $z_1$  and  $z_2$  are uniformly distributed over  $\mathbb{Z}_q$  since  $r_1, r_2$  are uniform. Given uniform  $z_1, z_2$  and a fixed  $c$ , the values  $C'$  and  $D'$  are uniquely determined and their distribution in the simulation matches the honest execution. Therefore, the simulated transcript is distributed identically to the honest one, and thus, special honest-verifier zero-knowledge holds.  $\square$

#### A.4 Batched Re-keying

Key rotation re-keys the four limb decryption handles of an account's active balance in place: each old handle  $D_i$  under  $\text{pk}_U$  is replaced by  $D'_i$  under the new key  $\text{pk}'_U$ , while the limb commitments are reused unchanged (Algorithm 8). The holder knows the quotient  $w = \text{sk}'_U \text{sk}_U^{-1}$  relating the two keys, and the same  $w$  transports the key and every handle:  $\text{pk}'_U = w \text{pk}_U$  and  $D'_i = w D_i$ . The re-keying proof  $\Pi_{\text{rekey}}$  (Figure 5) is a NIZK proof of knowledge for the relation

$$\mathcal{R}_{\text{rekey}} = \left\{ ((\text{pk}_U, \text{pk}'_U, (D_i, D'_i)_{i=0}^3); w) : \text{pk}'_U = w \text{pk}_U \wedge D'_i = w D_i \text{ for } i = 0, \dots, 3 \right\},$$

a single Chaum–Pedersen [CP93] discrete-log-equality statement batched over the five base/image pairs  $(\text{pk}_U, \text{pk}'_U)$  and  $(D_i, D'_i)_{i=0}^3$  under one shared witness  $w$ . We analyze  $\Sigma_{\text{rekey}}$ , the public-coin three-move  $\Sigma$ -protocol whose Fiat–Shamir form is shown in Figure 5: the interactive protocol replaces the hash-derived challenge with a verifier-sampled  $c \in \mathbb{Z}_q$ . We refer to the verifier check on the key pair as  $(V\star)$  and to the four checks on the handle pairs as  $(Vi)$  for  $i \in \{0, \dots, 3\}$ .

**Lemma 2.** *The protocol  $\Sigma_{\text{rekey}}$  is a public-coin 3-move  $\Sigma$ -protocol for the relation  $\mathcal{R}_{\text{rekey}}$ .*

*Proof.* To prove the lemma, we show completeness, 2-special soundness, and perfect special honest-verifier zero-knowledge.

*Perfect Completeness.* For an honest prover the response is  $z = s + cw$ , and using  $\text{pk}'_U = w \text{pk}_U$  and  $D'_i = w D_i$  the verification equations hold with probability 1 since:

- $(V\star) \ z \text{pk}_U = (s + cw) \text{pk}_U = s \text{pk}_U + c(w \text{pk}_U) = P' + c \text{pk}'_U$
- $(Vi) \ z D_i = (s + cw) D_i = s D_i + c(w D_i) = D''_i + c D'_i \text{ for } i \in \{0, \dots, 3\}$

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $\Pi_{\text{rekey}}.\text{Prove}(w; \text{sid}, \text{pk}_U, \text{pk}'_U, (D_i, D'_i)_{i=0}^3) :$ <hr/> <pre> 1 : <math>s \leftarrow \mathbb{Z}_q</math> 2 : <math>P' = s \text{pk}_U, \quad D''_i = s D_i \text{ for } i \in \{0, \dots, 3\}</math> 3 : <math>c = \text{H}(\text{sid}, \text{pk}_U, \text{pk}'_U, (D_i, D'_i)_{i=0}^3, P', (D''_i)_{i=0}^3)</math> 4 : <math>z = s + c w</math> 5 : <b>return</b> <math>\pi = (P', (D''_i)_{i=0}^3, z)</math> </pre><br>$\Pi_{\text{rekey}}.\text{Verify}(\pi; \text{sid}, \text{pk}_U, \text{pk}'_U, (D_i, D'_i)_{i=0}^3) :$ <hr/> <pre> 1 : <math>(P', (D''_i)_{i=0}^3, z) = \pi</math> 2 : <math>c = \text{H}(\text{sid}, \text{pk}_U, \text{pk}'_U, (D_i, D'_i)_{i=0}^3, P', (D''_i)_{i=0}^3)</math> 3 : <math>b_\star = (z \text{pk}_U \stackrel{?}{=} P' + c \text{pk}'_U)</math> 4 : <math>b_i = (z D_i \stackrel{?}{=} D''_i + c D'_i) \text{ for } i \in \{0, \dots, 3\}</math> 5 : <b>return</b> <math>b_\star \wedge b_0 \wedge b_1 \wedge b_2 \wedge b_3</math> </pre> |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Figure 5: NIZK batched re-keying DDH protocol  $\Pi_{\text{rekey}}$  (shared witness over five base/image pairs); proof size:  $5\mathbb{G} + \mathbb{F}$ .

Thus, completeness holds.

*2-Special Soundness.* Given two accepting transcripts  $(P', (D''_i)_{i=0}^3, c, z)$  and  $(P', (D''_i)_{i=0}^3, c', z')$  with matching first message but differing challenges  $c \neq c'$ , we show that there exists a PPT extractor  $\text{Ext}$  that can extract a witness  $w$  satisfying the relation. Since both transcripts satisfy  $(V\star)$ , subtracting  $z' \text{pk}_U = P' + c' \text{pk}'_U$  from  $z \text{pk}_U = P' + c \text{pk}'_U$  gives  $(z - z') \text{pk}_U = (c - c') \text{pk}'_U$ . Since  $c \neq c'$ , the value  $c - c'$  is invertible in  $\mathbb{Z}_q$  and the scalar  $w = (z - z')/(c - c')$  is well-defined and satisfies  $\text{pk}'_U = w \text{pk}_U$ . Applying the same subtraction to each  $(Vi)$  yields  $(z - z') D_i = (c - c') D'_i$ , hence  $D'_i = w D_i$  for the same scalar  $w$ , for all  $i \in \{0, \dots, 3\}$ . Thus  $\text{Ext}$  outputs a single  $w$  satisfying every clause of  $\mathcal{R}_{\text{rekey}}$ , and 2-special soundness holds.

*Perfect Special Honest-Verifier Zero-Knowledge.* Assuming the verifier is honest, we show that there exists a PPT simulator  $\mathcal{S}$  that, given the statement  $s$  and the challenge  $c$  in advance, produces a transcript  $(P', (D''_i)_{i=0}^3, c, z)$  that is identically distributed to that of an honest prover. The simulator samples  $z \in \mathbb{Z}_q$  uniformly at random and sets  $P' = z \text{pk}_U - c \text{pk}'_U$  and  $D''_i = z D_i - c D'_i$  for all  $i$ . By construction the simulated transcript satisfies  $(V\star)$  and  $(Vi)$ . In an honest execution the response  $z = s + c w$  is uniformly distributed over  $\mathbb{Z}_q$  since  $s$  is uniform, matching the simulator's choice. Conditioned on  $z$  and the fixed challenge  $c$ , the first message  $(P', (D''_i)_{i=0}^3)$  is uniquely determined by the verification equations and is identical in both worlds. Therefore, the simulated transcript is distributed identically to the honest one, and special honest-verifier zero-knowledge holds.  $\square$

By Lemma 2 and the Fiat-Shamir transform,  $\Pi_{\text{rekey}} = \text{FS}[\Sigma_{\text{rekey}}]$  is a NIZK proof of knowledge for  $\mathcal{R}_{\text{rekey}}$  in the random oracle model. The on-chain key-rotation check (Algorithm 8) pairs it with the assertion  $\text{pk}'_U \neq \mathcal{O}$  and carries no auditor material: rotation re-keys one token's balance handles and nothing else.

| $\Pi_{\mathbf{B}}^{\otimes}.\text{Prove}((\mathbf{x}_j)_j; \text{sid}, \mathbf{B}, (\mathbf{y}_j)_j) :$ | $\Pi_{\mathbf{B}}^{\otimes}.\text{Verify}(\pi; \text{sid}, \mathbf{B}, (\mathbf{y}_j)_j) :$          |
|---------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| 1 : $\mathbf{s} \leftarrow \mathbb{Z}_q^m; \mathbf{T} = \mathbf{B} \mathbf{s} \in \mathbb{G}^p$         | 1 : $(\mathbf{T}, \mathbf{z}) = \pi$                                                                 |
| 2 : $c = \text{H}(\text{sid}, \mathbf{B}, (\mathbf{y}_j)_j, \mathbf{T})$                                | 2 : $c = \text{H}(\text{sid}, \mathbf{B}, (\mathbf{y}_j)_j, \mathbf{T})$                             |
| 3 : $\mathbf{z} = \mathbf{s} + \sum_{j=1}^n c^j \mathbf{x}_j \in \mathbb{Z}_q^m$                        | 3 : <b>return</b> $\mathbf{B} \mathbf{z} \stackrel{?}{=} \mathbf{T} + \sum_{j=1}^n c^j \mathbf{y}_j$ |
| 4 : <b>return</b> $\pi = (\mathbf{T}, \mathbf{z})$                                                      |                                                                                                      |

Figure 6: Witness-fold NIZKPoK  $\Pi_{\mathbf{B}}^{\otimes}$  for the tensor language  $\mathbf{I}_n \otimes \mathbf{B}$  with block  $\mathbf{B} \in \mathbb{G}^{p \times m}$ ; proof size  $p\mathbb{G} + m\mathbb{F}$ , independent of the number of instances  $n$ .

## B Witness-Folding: Batching Schnorr Proofs over a Shared Base Block

Many protocol operations emit several  $\Sigma$ -proofs over the same public bases: a transfer proves one ElGamal consistency per receiver under a common ciphertext layout, unwrap and balance updates prove a fixed set of same-key consistencies, and so on. Written out,  $n$  such proofs over one base block cost  $n$  times a single proof. This appendix shows that instances sharing a base block collapse to one proof whose size does not grow with  $n$ .

**Setup.** View  $\mathbb{G}$  (prime order  $q$ , written additively) as a  $\mathbb{Z}_q$ -module and extend the scalar action entrywise to matrices, so a base block  $\mathbf{B} \in \mathbb{G}^{p \times m}$  acts as a  $\mathbb{Z}_q$ -linear map  $\mathbf{B} : \mathbb{Z}_q^m \rightarrow \mathbb{G}^p, \mathbf{x} \mapsto \mathbf{B}\mathbf{x}$ . A single Schnorr instance is a witness  $\mathbf{x} \in \mathbb{Z}_q^m$  with  $\mathbf{B}\mathbf{x} = \mathbf{y}$  for a public target  $\mathbf{y} \in \mathbb{G}^p$ ; its generalized-Schnorr proof has size  $p\mathbb{G} + m\mathbb{F}$ .

**The tensor language.** Fix  $\mathbf{B} \in \mathbb{G}^{p \times m}$  and take  $n$  instances sharing it: witnesses  $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{Z}_q^m$  and targets  $\mathbf{y}_j = \mathbf{B}\mathbf{x}_j$ . Stacking them gives the block-diagonal Kronecker system

$$(\mathbf{I}_n \otimes \mathbf{B}) \mathbf{x} = \mathbf{y}, \quad \mathbf{x} = (\mathbf{x}_1, \dots, \mathbf{x}_n) \in \mathbb{Z}_q^{nm}, \quad \mathbf{y} = (\mathbf{y}_1, \dots, \mathbf{y}_n) \in \mathbb{G}^{np},$$

with relation

$$\mathcal{R}_{\mathbf{B}}^{\otimes} = \{(\mathbf{B}, (\mathbf{y}_j)_{j=1}^n; (\mathbf{x}_j)_{j=1}^n) : \mathbf{y}_j = \mathbf{B}\mathbf{x}_j \text{ for all } j\}.$$

Proving the instances one by one costs  $n(p\mathbb{G} + m\mathbb{F})$ . The witness-fold instead combines them along their witnesses (the columns of the Kronecker system): for a challenge  $c$ , form the aggregate witness and target

$$\mathbf{x}^* = \sum_{j=1}^n c^j \mathbf{x}_j \in \mathbb{Z}_q^m, \quad \mathbf{y}^* = \sum_{j=1}^n c^j \mathbf{y}_j \in \mathbb{G}^p,$$

which satisfy  $\mathbf{B}\mathbf{x}^* = \mathbf{y}^*$  by linearity. A single generalized-Schnorr proof for the aggregate instance  $(\mathbf{B}, \mathbf{y}^*)$  then certifies the whole batch. Drawing  $c$  from the random oracle on the statement prevents the prover from choosing a batch the aggregate would mask. The Fiat–Shamir proof  $\Pi_{\mathbf{B}}^{\otimes}$  is given in Figure 6; its size is  $p\mathbb{G} + m\mathbb{F}$ , independent of  $n$ .

**Lemma 3** (Witness-fold is an unconditional NIZKPoK). *Let  $\mathbf{B} \in \mathbb{G}^{p \times m}$  be any base block over  $\mathbb{G}$  (prime order  $q$ ) and  $\text{H}$  a random oracle. Then  $\Pi_{\mathbf{B}}^{\otimes}$  (Figure 6) is a complete, special honest-verifier zero-knowledge,  $(n+1)$ -special-sound Fiat–Shamir NIZK proof of knowledge for  $\mathcal{R}_{\mathbf{B}}^{\otimes}$ .*

*Proof.* Throughout,  $\mathbf{B}$  is a fixed  $\mathbb{Z}_q$ -linear map  $\mathbb{Z}_q^m \rightarrow \mathbb{G}^p$ .

*Completeness.* For the honest response  $\mathbf{z} = \mathbf{s} + \sum_j c^j \mathbf{x}_j$ , linearity gives  $\mathbf{Bz} = \mathbf{Bs} + \sum_j c^j \mathbf{Bx}_j = \mathbf{T} + \sum_j c^j \mathbf{y}_j$ , using  $\mathbf{y}_j = \mathbf{Bx}_j$ .

*SHVZK.* On challenge  $c$ , the simulator draws  $\mathbf{z} \leftarrow \mathbb{Z}_q^m$ , sets  $\mathbf{T} := \mathbf{Bz} - \sum_j c^j \mathbf{y}_j$ , and programs  $\mathbf{H}(\text{sid}, \mathbf{B}, (\mathbf{y}_j)_j, \mathbf{T}) = c$ . The transcript verifies by construction. In a real run  $\mathbf{z} = \mathbf{s} + \sum_j c^j \mathbf{x}_j$  is uniform on  $\mathbb{Z}_q^m$  (since  $\mathbf{s}$  is), and the honest  $\mathbf{T} = \mathbf{Bs} = \mathbf{Bz} - \sum_j c^j \mathbf{y}_j$  is the same function of  $(\mathbf{z}, \text{statement})$  as in the simulation, so the two distributions are identical.

*(n+1)-special soundness.* Fix a first message  $\mathbf{T}$  and take  $n+1$  accepting transcripts with pairwise-distinct challenges  $c_0, \dots, c_n$  and responses  $\mathbf{z}_0, \dots, \mathbf{z}_n \in \mathbb{Z}_q^m$ . The accepting equations

$$\mathbf{Bz}_i = \mathbf{T} + \sum_{j=1}^n c_i^j \mathbf{y}_j \in \mathbb{G}^p \quad (i = 0, \dots, n)$$

state that the group elements  $\mathbf{Bz}_i$  are the values at  $c_i$  of the degree- $\leq n$  polynomial whose coefficients are  $\mathbf{T}, \mathbf{y}_1, \dots, \mathbf{y}_n$  ( $\mathbf{T}$  is the degree-0 term). The nodes  $c_i$  are distinct, so the Vandermonde matrix  $V = (c_i^j)_{0 \leq i, j \leq n}$  is invertible over  $\mathbb{Z}_q$ ; reading off the degree- $j$  coefficient and using linearity of  $\mathbf{B}$  yields, for  $1 \leq j \leq n$ ,

$$\mathbf{y}_j = \sum_i (V^{-1})_{ji} \mathbf{Bz}_i = \mathbf{Bx}_j, \quad \mathbf{x}_j := \sum_i (V^{-1})_{ji} \mathbf{z}_i \in \mathbb{Z}_q^m.$$

Thus  $(\mathbf{x}_j)_{j=1}^n$  is a witness for  $\mathcal{R}_{\mathbf{B}}^\otimes$ . The extractor computes it by solving the  $\mathbb{Z}_q$ -Vandermonde system on the responses, and its correctness uses only that  $\mathbf{B}$  is  $\mathbb{Z}_q$ -linear (evaluated forward, as in  $\sum_i (V^{-1})_{ji} \mathbf{Bz}_i = \mathbf{B} \sum_i (V^{-1})_{ji} \mathbf{z}_i$ ). No discrete log is ever taken, so the argument is unaffected by any discrete-log relation among the entries of  $\mathbf{B}$ .  $\square$

In  $\Pi_{\text{ConfTx}}$ ,  $n \in \{4, 5\}$  per key and  $n = 2K \leq 510$  for  $\gamma_{\text{aud}}$ . With  $|\mathcal{C}| = 2^{248}$ , the Fiat-Shamir knowledge error is at most  $(Q_H + 1)n/|\mathcal{C}| \leq (Q_H + 1)2^{-239}$  [AFK22], and extraction solves one  $(n+1) \times (n+1)$  Vandermonde system over  $\mathbb{Z}_q$ , with no discrete logarithms.

**Application: DDH tuples ( $2\mathbb{G} + \mathbb{F}$ ).** Take  $\mathbf{B} = \begin{pmatrix} E \\ F \end{pmatrix}$  ( $p = 2, m = 1$ ) and targets  $\mathbf{y}_j = (X_j, Y_j)^\top$  with  $X_j = x_j E, Y_j = x_j F$ . Writing  $\mathbf{T} = (X', Y')^\top = (sE, sF)^\top$  and  $\mathbf{z} = z$ , the proof  $\pi = (X', Y', z) \in 2\mathbb{G} + \mathbb{F}$  has the two checks

$$zE \stackrel{?}{=} X' + \sum_j c^j X_j \quad (\text{V1}), \quad zF \stackrel{?}{=} Y' + \sum_j c^j Y_j \quad (\text{V2}).$$

Lemma 3 extracts a single scalar per tuple,  $x_j = \sum_i (V^{-1})_{ji} z_i$ , which satisfies the whole relation  $\mathbf{Bx}_j = \mathbf{y}_j$ , that is  $X_j = x_j E$  (from V1) and  $Y_j = x_j F$  (from V2) with the one common  $x_j$ . Each  $(X_j, Y_j)$  is therefore a genuine DDH tuple, and this holds even when the prover knows  $\log_E F$ .

**Application: Same-key ElGamal ( $2\mathbb{G} + 2\mathbb{F}$ ).** Take  $\mathbf{B} = \text{eg}(\text{pk}) = \begin{pmatrix} G & H \\ \text{pk} & 0 \end{pmatrix}$  ( $p = 2, m = 2$ ) and  $n$  ciphertexts  $(C_j, D_j)$  under one key  $\text{pk}$ , with witnesses  $\mathbf{x}_j = (r_j, v_j)$ . Writing  $\mathbf{T} = (C', D')^\top = (aG + bH, \text{apk})^\top$  and  $\mathbf{z} = (z_1, z_2)$ ,

$$a, b \leftarrow \mathbb{Z}_q; \quad z_1 = a + \sum_j c^j r_j, \quad z_2 = b + \sum_j c^j v_j; \quad c = \mathbf{H}(\text{sid}, \text{pk}, (C_j, D_j)_j, C', D'); \quad \pi = (C', D', z_1, z_2),$$

$$z_1 G + z_2 H \stackrel{?}{=} C' + \sum_j c^j C_j \quad (\text{V1}), \quad z_1 \text{pk} \stackrel{?}{=} D' + \sum_j c^j D_j \quad (\text{V2}),$$

of size  $2\mathbb{G} + 2\mathbb{F}$ . The extractor returns  $r_j = \sum_i (V^{-1})_{ji} z_1^{(i)}$  and  $v_j = \sum_i (V^{-1})_{ji} z_2^{(i)}$  from (V1), and  $D_j = r_j \text{pk}$  from (V2) via the shared  $z_1^{(i)}$ . Because both checks are kept, the proof stays sound even though a holder knows its own  $\text{sk} = \log_G \text{pk}$ ; this is the base relation covered by Lemma 3.

| Algorithm                       | Witness-fold batches                                                                                                    | Standalone           | Unoptimized size                          | Witness-fold optimized                |
|---------------------------------|-------------------------------------------------------------------------------------------------------------------------|----------------------|-------------------------------------------|---------------------------------------|
| 1. Register (Alg. 1)            | —                                                                                                                       | —                    | —                                         | —                                     |
| 2. Auditor update (Alg. 2)      | —                                                                                                                       | —                    | —                                         | —                                     |
| 3. Wrap                         | —                                                                                                                       | —                    | —                                         | —                                     |
| 4–5. Transfer (Alg. 4, 5)       | $\mathbf{eg}_{\mathbf{pk}_S}^{(5)}, \mathbf{eg}_{\mathbf{pk}_{R_k}}^{(4)} \times K, \mathbf{eg}_{\mathbf{pk}_*}^{(2K)}$ | $\pi_{\text{bal}}$   | $(12K+12)\mathbb{G} + (12K+11)\mathbb{F}$ | $(2K+6)\mathbb{G} + (2K+5)\mathbb{F}$ |
| 6. Unwrap (Alg. 6)              | $\mathbf{eg}_{\mathbf{pk}_{t'}}^{(4)}$                                                                                  | $\pi_{\text{bal}}$   | $10\mathbb{G} + 9\mathbb{F}$              | $4\mathbb{G} + 3\mathbb{F}$           |
| 7. Merge / update bal. (Alg. 7) | $\mathbf{eg}_{\mathbf{pk}_{t'}}^{(4)}$                                                                                  | $\pi_{\text{eq}}$    | $10\mathbb{G} + 9\mathbb{F}$              | $4\mathbb{G} + 3\mathbb{F}$           |
| 8. Key rotation (Alg. 8)        | —                                                                                                                       | $\Pi_{\text{rekey}}$ | $5\mathbb{G} + \mathbb{F}$                | $5\mathbb{G} + \mathbb{F}$            |

Table 1: Witness-fold optimization per operation. Consistency proofs sharing one key form a tensor  $\mathbf{I}_t \otimes \mathbf{eg}(\mathbf{pk})$  and batch to  $2\mathbb{G} + 2\mathbb{F}$  independent of their count (Lemma 3); this holds even though holders know their own  $\mathbf{sk}$ , since both ElGamal checks are retained. Each balance/equality DDH is a lone tuple and stays at  $2\mathbb{G} + \mathbb{F}$ . A transfer’s  $2K$  auditor chunks share the key  $\mathbf{pk}_*$  and so form one further batch  $\mathbf{eg}_{\mathbf{pk}_*}^{(2K)}$ , which the fold collapses to  $2\mathbb{G} + 2\mathbb{F}$  regardless of the number of receivers in the batch; unfolded, they would cost  $2K$  separate proofs.  $\Pi_{\text{rekey}}$  is not a same-key  $\mathbf{eg}$  tensor and is unchanged.  $K$  is the number of transfer receivers; the unoptimized size sums the constituent proofs (Appendix A). Range proofs are unaffected by the fold and excluded from both size columns.

**One batched proof per operation.** Table 1 applies the witness-fold to each operation’s proof bundle. Consistency proofs sharing one key form a tensor  $\mathbf{I}_t \otimes \mathbf{eg}(\mathbf{pk})$  and batch to  $2\mathbb{G} + 2\mathbb{F}$  regardless of their count  $t$  (Lemma 3); we write  $\mathbf{eg}_{\mathbf{pk}}^{(t)}$  for such a batch. The balance and equality proofs are lone DDH tuples (one per operation), so there is nothing to batch and each stays at its standalone  $2\mathbb{G} + \mathbb{F}$ . A transfer’s auditor chunks are same-key ciphertexts under  $\mathbf{pk}_*$  like any other, so they fold as one further batch  $\mathbf{eg}_{\mathbf{pk}_*}^{(2K)}$  covering all  $2K$  chunks of the receiver batch at  $2\mathbb{G} + 2\mathbb{F}$  (Section 3.3).  $\Pi_{\text{rekey}}$  does not admit witness-folding and is left unchanged.

## C DDH or ElGamal Consistency in the Transfer Proofs

**Two base blocks.** Every non-range proof of a transfer is a linear statement over a base block (Appendix B), and two blocks cover all of them: DDH, with block  $\begin{pmatrix} E \\ F \end{pmatrix}$  and one witness, of size  $2\mathbb{G} + \mathbb{F}$ ; and EG(pk), with block  $\begin{pmatrix} G & H \\ \text{pk} & 0 \end{pmatrix}$  and two witnesses, of size  $2\mathbb{G} + 2\mathbb{F}$ . Which applies is decided by the message. If the message is public, or zero by construction, the statement has a single witness and DDH suffices; if it is hidden, the second column is unavoidable. Relative cost is decided by batching: instances of one block witness-fold into a single proof of unchanged size at any count, while instances of different blocks do not fold at all. Three sites admit more than one sound choice; they are compared in Table 2.

**What the balance proof leaves open.** The balance relation is the case where the message vanishes. Here  $\Delta$  encrypts zero, the witness  $\text{sk}_S$  is the only one left, and  $\Pi_{\text{DDH}}$  at  $2\mathbb{G} + \mathbb{F}$  is the right shape. What  $\pi_{\text{bal}}$  does not do is constrain the handles  $\Delta$  is built from: a prover holding  $\text{sk}_S$  can meet  $\Delta.D = \text{sk}_S \Delta.C$  for any  $\Delta.C$  at all by setting  $\Delta.D$  that way, so the proof ties the two halves of  $\Delta$  together and says nothing about how  $\Delta$  splits across the ciphertexts forming it. Supplying that missing constraint is the job of  $\gamma_S$ , and the freedom it closes off is specific: a holder who knows  $\text{sk}$  can post  $D = r\text{pk} + \delta \text{sk}H$ , which decrypts to  $(v - \delta)H$  while remaining consistent with  $\text{sk}$ . Only the two coupled EG equations, which share a response, exclude such a  $\delta$ .

**The new balance.** Without the four  $B'_S$  slots, a sender with  $b_S = 0$  posts an honest  $A_R = (g^r h^a, \text{pk}_R^r)$  and  $\hat{A}_S = (g^r h^a, \text{pk}_S^r)$ , and sets  $B'_S = (g^{r'}, \text{pk}_S^{r'} h^{\text{sk}_S a})$ : the commitment opens to 0 and the range proof passes, while the ciphertext decrypts to  $-a$  and  $\pi_{\text{bal}}$  passes too, so the receiver is credited  $a$  from nothing. The range proof binds the commitments, every later operation reads the ciphertext, and the EG slots are the only link between them. Soundness needs one slot, on  $\text{Collapse}(B'_S)$ , since Pedersen binding then forces the collapsed opening below  $2^{64}$ . The four limbs are proven because well-formedness is one uniform predicate over an encrypted amount (Definition 1) and the fold makes the extra instances free; nothing in the security argument distinguishes the two. The corresponding slots on  $A_{R_k}$  are not optional, since an honest receiver must decrypt those limbs.

**The aggregate sender handle.** Without its slot the same shift is cheaper still: with  $b'_S = b_S$  and  $\hat{A}_S = (g^r h^\delta, \text{pk}_S^r h^{\text{sk}_S \delta})$ , the debit decrypts to zero while the receiver is credited  $\delta$ . A DDH proof is nonetheless expressible when  $K = 1$ , since  $\gamma_R$  pins  $r$  as the blinding of  $\text{Collapse}(A_R).C$  and a DDH on  $(\text{pk}_R, \text{pk}_S)$  then forces  $\hat{A}_S.D = \text{pk}_S^r$ . It is sound but not worth taking, because  $\gamma_S$  is  $2\mathbb{G} + 2\mathbb{F}$  whether or not it covers  $\hat{A}_S$ , so the slot costs nothing and the DDH is a pure addition. It also does not generalize: for  $K > 1$  the aggregate exponent  $r_S = \sum_k \rho_k$  is spread over distinct receiver keys, and no single public base carries it.

**The auditor handles.** The auditor handles of a transfer (Section 3.3) sit differently, because the sender does not know  $\text{sk}_{\text{aud}}$  and cannot form the shift above. Nothing can be inflated; a bad handle only makes the amount unreadable, so the choice is one of cost. The adopted design proves one  $\text{EG}(\text{pk}_{\text{aud}})$  batch over all  $2K$  chunk ciphertexts, which is  $2\mathbb{G} + 2\mathbb{F}$  at any  $K$ . A DDH against the receiver-side chunk handle  $\text{pk}_{R_k}^{\hat{\rho}_{k,j}}$ , which the ledger also recomputes, is equally sound, but its block varies with  $k$  and so costs  $K(2\mathbb{G} + \mathbb{F})$ . Counting  $\mathbb{G}$  and  $\mathbb{F}$  alike at 32 bytes and including the  $2K$  posted handles, the two come to  $2K + 4$  and  $5K$  units: the DDH is smaller by one unit at  $K = 1$  and larger from  $K = 2$  on.

| Site                  | Adopted                                                                                            | Alternative                                                                                      | Why not the alternative                                                                                                                     |
|-----------------------|----------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| New balance<br>$B'_S$ | EG on all four limbs,<br>inside $\gamma_S$                                                         | EG on $\text{Collapse}(B'_S)$<br>alone, inside $\gamma_S$                                        | Equally sound, at the same size;<br>the four-limb form is just the<br>uniform well-formedness check                                         |
| Aggregate $\hat{A}_S$ | one further slot inside $\gamma_S$                                                                 | DDH on $(\mathbf{pk}_R, \mathbf{pk}_S)$ ,<br>$2\mathbb{G}+\mathbb{F}$ , at $K=1$ only            | $\gamma_S$ is $2\mathbb{G}+2\mathbb{F}$ either way, so the<br>slot is free and the DDH pure<br>addition; no such base exists for<br>$K > 1$ |
| Auditor<br>handles    | one $\text{EG}(\mathbf{pk}_{\text{aud}})$ batch over<br>all $2K$ chunks, $2\mathbb{G}+2\mathbb{F}$ | DDH per receiver<br>against $\mathbf{pk}_{R_k}^{\hat{p}_{k,j}}$ ,<br>$K(2\mathbb{G}+\mathbb{F})$ | Equally sound, and smaller by one<br>unit at $K=1$ , but larger from<br>$K=2$ on                                                            |

Table 2: The three sites where more than one sound design exists, for a transfer with  $K$  receivers. Sizes are proof material only: the  $2K$  auditor handles are posted under either auditor option and so do not separate them, and range proofs are excluded as in Table 1. “Inside  $\gamma_S$ ” means the instance rides the existing witness-fold under  $\mathbf{pk}_S$ , which is  $2\mathbb{G} + 2\mathbb{F}$  at any batch size and so is unchanged by adding it.